

## Programos v3.0 dokumentacija

Generated by Doxygen 1.13.2

|                                                |          |
|------------------------------------------------|----------|
| <b>1 Hierarchical Index</b>                    | <b>3</b> |
| 1.1 Class Hierarchy                            | 3        |
| <b>2 Class Index</b>                           | <b>5</b> |
| 2.1 Class List                                 | 5        |
| <b>3 File Index</b>                            | <b>7</b> |
| 3.1 File List                                  | 7        |
| <b>4 Class Documentation</b>                   | <b>9</b> |
| 4.1 Stud Class Reference                       | 9        |
| 4.1.1 Constructor & Destructor Documentation   | 10       |
| 4.1.1.1 Stud() [1/4]                           | 10       |
| 4.1.1.2 Stud() [2/4]                           | 10       |
| 4.1.1.3 Stud() [3/4]                           | 10       |
| 4.1.1.4 Stud() [4/4]                           | 10       |
| 4.1.1.5 ~Stud()                                | 11       |
| 4.1.2 Member Function Documentation            | 11       |
| 4.1.2.1 clear()                                | 11       |
| 4.1.2.2 galutinis()                            | 11       |
| 4.1.2.3 galutinisBalas()                       | 11       |
| 4.1.2.4 getEgzaminas()                         | 11       |
| 4.1.2.5 getND()                                | 11       |
| 4.1.2.6 getPaz()                               | 11       |
| 4.1.2.7 grazintiPaskutini()                    | 11       |
| 4.1.2.8 operator"!=(                           | 11       |
| 4.1.2.9 operator=(                             | 12       |
| 4.1.2.10 operator=(                            | 12       |
| 4.1.2.11 operator==(                           | 12       |
| 4.1.2.12 pazKiekis()                           | 12       |
| 4.1.2.13 removeLast()                          | 12       |
| 4.1.2.14 removeND()                            | 12       |
| 4.1.2.15 setBalasGalutinisVid()                | 12       |
| 4.1.2.16 setEgzaminas()                        | 12       |
| 4.1.2.17 setND()                               | 12       |
| 4.1.2.18 setPavarde()                          | 13       |
| 4.1.2.19 setVardas()                           | 13       |
| 4.1.3 Friends And Related Symbol Documentation | 13       |
| 4.1.3.1 operator<<                             | 13       |
| 4.1.3.2 operator>>                             | 13       |
| 4.2 Vector< T > Class Template Reference       | 13       |
| 4.2.1 Constructor & Destructor Documentation   | 14       |
| 4.2.1.1 Vector() [1/5]                         | 14       |
| 4.2.1.2 Vector() [2/5]                         | 15       |

|                                               |    |
|-----------------------------------------------|----|
| 4.2.1.3 Vector() [3/5] . . . . .              | 15 |
| 4.2.1.4 ~Vector() . . . . .                   | 15 |
| 4.2.1.5 Vector() [4/5] . . . . .              | 15 |
| 4.2.1.6 Vector() [5/5] . . . . .              | 15 |
| 4.2.2 Member Function Documentation . . . . . | 15 |
| 4.2.2.1 append_range() . . . . .              | 15 |
| 4.2.2.2 assign() [1/3] . . . . .              | 15 |
| 4.2.2.3 assign() [2/3] . . . . .              | 16 |
| 4.2.2.4 assign() [3/3] . . . . .              | 16 |
| 4.2.2.5 at() . . . . .                        | 16 |
| 4.2.2.6 back() . . . . .                      | 16 |
| 4.2.2.7 begin() . . . . .                     | 16 |
| 4.2.2.8 Capacity() . . . . .                  | 16 |
| 4.2.2.9 cbegin() . . . . .                    | 16 |
| 4.2.2.10 cend() . . . . .                     | 16 |
| 4.2.2.11 clear() . . . . .                    | 17 |
| 4.2.2.12 crbegin() . . . . .                  | 17 |
| 4.2.2.13 crend() . . . . .                    | 17 |
| 4.2.2.14 Data() . . . . .                     | 17 |
| 4.2.2.15 empty() . . . . .                    | 17 |
| 4.2.2.16 end() . . . . .                      | 17 |
| 4.2.2.17 erase() [1/2] . . . . .              | 17 |
| 4.2.2.18 erase() [2/2] . . . . .              | 17 |
| 4.2.2.19 front() . . . . .                    | 18 |
| 4.2.2.20 insert() . . . . .                   | 18 |
| 4.2.2.21 max_size() . . . . .                 | 18 |
| 4.2.2.22 operator!=(()) . . . . .             | 18 |
| 4.2.2.23 operator<() . . . . .                | 18 |
| 4.2.2.24 operator<=() . . . . .               | 18 |
| 4.2.2.25 operator=() [1/2] . . . . .          | 18 |
| 4.2.2.26 operator=() [2/2] . . . . .          | 18 |
| 4.2.2.27 operator==(()) . . . . .             | 19 |
| 4.2.2.28 operator>() . . . . .                | 19 |
| 4.2.2.29 operator>=() . . . . .               | 19 |
| 4.2.2.30 operator[]() [1/2] . . . . .         | 19 |
| 4.2.2.31 operator[]() [2/2] . . . . .         | 19 |
| 4.2.2.32 pop_back() . . . . .                 | 19 |
| 4.2.2.33 push_back() . . . . .                | 19 |
| 4.2.2.34 rbegin() . . . . .                   | 19 |
| 4.2.2.35 rend() . . . . .                     | 20 |
| 4.2.2.36 reserve() . . . . .                  | 20 |
| 4.2.2.37 resize() [1/2] . . . . .             | 20 |

|          |                                        |           |
|----------|----------------------------------------|-----------|
| 4.2.2.38 | resize() [2/2]                         | 20        |
| 4.2.2.39 | shrink_to_fit()                        | 20        |
| 4.2.2.40 | Size()                                 | 20        |
| 4.2.2.41 | swap()                                 | 20        |
| 4.3      | zmogus Class Reference                 | 21        |
| 4.3.1    | Constructor & Destructor Documentation | 21        |
| 4.3.1.1  | zmogus() [1/3]                         | 21        |
| 4.3.1.2  | zmogus() [2/3]                         | 21        |
| 4.3.1.3  | ~zmogus()                              | 21        |
| 4.3.1.4  | zmogus() [3/3]                         | 22        |
| 4.3.2    | Member Function Documentation          | 22        |
| 4.3.2.1  | getPavarde()                           | 22        |
| 4.3.2.2  | getVardas()                            | 22        |
| 4.3.2.3  | operator=()                            | 22        |
| 4.3.2.4  | setPavarde()                           | 22        |
| 4.3.2.5  | setVardas()                            | 22        |
| 4.3.3    | Member Data Documentation              | 22        |
| 4.3.3.1  | pavarde                                | 22        |
| 4.3.3.2  | vardas                                 | 22        |
| <b>5</b> | <b>File Documentation</b>              | <b>23</b> |
| 5.1      | functions.cpp File Reference           | 23        |
| 5.1.1    | Function Documentation                 | 24        |
| 5.1.1.1  | constrTest()                           | 24        |
| 5.1.1.2  | copyAssignTest()                       | 24        |
| 5.1.1.3  | copyConstrTest()                       | 24        |
| 5.1.1.4  | destructorTest()                       | 24        |
| 5.1.1.5  | duomenu_generavimas()                  | 24        |
| 5.1.1.6  | duomenu_ivedimas()                     | 24        |
| 5.1.1.7  | filePasirinkimas()                     | 24        |
| 5.1.1.8  | GeneruotiFiles()                       | 24        |
| 5.1.1.9  | inputTest()                            | 24        |
| 5.1.1.10 | konstruktoriuTest()                    | 25        |
| 5.1.1.11 | moveAssignTest()                       | 25        |
| 5.1.1.12 | moveConstTest()                        | 25        |
| 5.1.1.13 | operator<<()                           | 25        |
| 5.1.1.14 | operator>>()                           | 25        |
| 5.1.1.15 | outputTest()                           | 25        |
| 5.1.1.16 | PalygintiKategorijas()                 | 25        |
| 5.1.1.17 | PalygintiPavardes()                    | 25        |
| 5.1.1.18 | PalygintiVardas()                      | 26        |
| 5.1.1.19 | paz_gener()                            | 26        |
| 5.1.1.20 | pazymiu_ivedimas()                     | 26        |

|                                 |    |
|---------------------------------|----|
| 5.1.1.21 print()                | 26 |
| 5.1.1.22 printCases()           | 26 |
| 5.1.1.23 pushbackTest()         | 26 |
| 5.1.1.24 tikrinimas()           | 26 |
| 5.1.1.25 tinkamas_char()        | 26 |
| 5.1.1.26 tinkamas_int()         | 27 |
| 5.1.1.27 vardai()               | 27 |
| 5.2 functions.h File Reference  | 27 |
| 5.2.1 Function Documentation    | 28 |
| 5.2.1.1 duomenu_generavimas()   | 28 |
| 5.2.1.2 duomenu_ivedimas()      | 28 |
| 5.2.1.3 filePasirinkimas()      | 28 |
| 5.2.1.4 fileskait()             | 28 |
| 5.2.1.5 GeneruotiFiles()        | 28 |
| 5.2.1.6 konstruktoriuTest()     | 28 |
| 5.2.1.7 PalygintiKategorijas()  | 28 |
| 5.2.1.8 PalygintiPavardes()     | 28 |
| 5.2.1.9 PalygintiVardas()       | 29 |
| 5.2.1.10 paz_gener()            | 29 |
| 5.2.1.11 pazymiu_ivedimas()     | 29 |
| 5.2.1.12 printCases()           | 29 |
| 5.2.1.13 PrintVektorius()       | 29 |
| 5.2.1.14 pushbackTest()         | 29 |
| 5.2.1.15 tikrinimas()           | 29 |
| 5.2.1.16 tinkamas_char()        | 30 |
| 5.2.1.17 tinkamas_int()         | 30 |
| 5.2.1.18 tyrimai()              | 30 |
| 5.2.1.19 vardai()               | 30 |
| 5.2.1.20 vectorIdejimas()       | 30 |
| 5.3 functions.h                 | 30 |
| 5.4 main.cpp File Reference     | 34 |
| 5.4.1 Function Documentation    | 34 |
| 5.4.1.1 main()                  | 34 |
| 5.5 main.h File Reference       | 35 |
| 5.5.1 Typedef Documentation     | 35 |
| 5.5.1.1 konteinerisDeque        | 35 |
| 5.5.1.2 konteinerisList         | 35 |
| 5.5.1.3 konteinerisVector       | 35 |
| 5.6 main.h                      | 36 |
| 5.7 strategija.h File Reference | 38 |
| 5.7.1 Function Documentation    | 38 |
| 5.7.1.1 nuskaitymasFile()       | 38 |

|                                         |    |
|-----------------------------------------|----|
| 5.7.1.2 studentuIsskirstymas()          | 39 |
| 5.7.1.3 studentuIsskirstymas2()         | 39 |
| 5.7.1.4 studentuRusiavimas()            | 39 |
| 5.7.1.5 studentuRusiavimas2()           | 39 |
| 5.7.1.6 StudentuRusiavimas3()           | 39 |
| 5.7.1.7 studentuSkirstymas3()           | 39 |
| 5.7.1.8 testavimasPrint()               | 40 |
| 5.7.1.9 vykdomaPrograma()               | 40 |
| 5.7.1.10 vykdomaPrograma2()             | 40 |
| 5.7.1.11 vykdomaPrograma3()             | 40 |
| 5.8 strategija.h                        | 40 |
| 5.9 Strategija1.cpp File Reference      | 45 |
| 5.9.1 Function Documentation            | 45 |
| 5.9.1.1 main()                          | 45 |
| 5.10 Strategija2.cpp File Reference     | 46 |
| 5.10.1 Function Documentation           | 46 |
| 5.10.1.1 main()                         | 46 |
| 5.11 Strategija3.cpp File Reference     | 46 |
| 5.11.1 Function Documentation           | 46 |
| 5.11.1.1 main()                         | 46 |
| 5.12 unitTesting.cpp File Reference     | 46 |
| 5.12.1 Macro Definition Documentation   | 47 |
| 5.12.1.1 CATCH_CONFIG_MAIN              | 47 |
| 5.12.2 Function Documentation           | 47 |
| 5.12.2.1 TEST_CASE() [1/7]              | 47 |
| 5.12.2.2 TEST_CASE() [2/7]              | 47 |
| 5.12.2.3 TEST_CASE() [3/7]              | 47 |
| 5.12.2.4 TEST_CASE() [4/7]              | 47 |
| 5.12.2.5 TEST_CASE() [5/7]              | 47 |
| 5.12.2.6 TEST_CASE() [6/7]              | 48 |
| 5.12.2.7 TEST_CASE() [7/7]              | 48 |
| 5.13 vector.h File Reference            | 48 |
| 5.14 vector.h                           | 48 |
| 5.15 vectorUnitTests.cpp File Reference | 52 |
| 5.15.1 Macro Definition Documentation   | 53 |
| 5.15.1.1 CATCH_CONFIG_MAIN              | 53 |
| 5.15.2 Function Documentation           | 53 |
| 5.15.2.1 TEST_CASE() [1/20]             | 53 |
| 5.15.2.2 TEST_CASE() [2/20]             | 53 |
| 5.15.2.3 TEST_CASE() [3/20]             | 53 |
| 5.15.2.4 TEST_CASE() [4/20]             | 54 |
| 5.15.2.5 TEST_CASE() [5/20]             | 54 |

|                               |    |
|-------------------------------|----|
| 5.15.2.6 TEST_CASE() [6/20]   | 54 |
| 5.15.2.7 TEST_CASE() [7/20]   | 54 |
| 5.15.2.8 TEST_CASE() [8/20]   | 54 |
| 5.15.2.9 TEST_CASE() [9/20]   | 54 |
| 5.15.2.10 TEST_CASE() [10/20] | 54 |
| 5.15.2.11 TEST_CASE() [11/20] | 54 |
| 5.15.2.12 TEST_CASE() [12/20] | 55 |
| 5.15.2.13 TEST_CASE() [13/20] | 55 |
| 5.15.2.14 TEST_CASE() [14/20] | 55 |
| 5.15.2.15 TEST_CASE() [15/20] | 55 |
| 5.15.2.16 TEST_CASE() [16/20] | 55 |
| 5.15.2.17 TEST_CASE() [17/20] | 55 |
| 5.15.2.18 TEST_CASE() [18/20] | 55 |
| 5.15.2.19 TEST_CASE() [19/20] | 55 |
| 5.15.2.20 TEST_CASE() [20/20] | 55 |

**Index****57**





# Chapter 1

## Hierarchical Index

### 1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

|                       |    |
|-----------------------|----|
| Vector< T > . . . . . | 13 |
| zmogus . . . . .      | 21 |
| Stud . . . . .        | 9  |



## Chapter 2

# Class Index

### 2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

|                                             |                    |
|---------------------------------------------|--------------------|
| <a href="#">Stud</a> . . . . .              | <a href="#">9</a>  |
| <a href="#">Vector&lt; T &gt;</a> . . . . . | <a href="#">13</a> |
| <a href="#">zmogus</a> . . . . .            | <a href="#">21</a> |



# Chapter 3

## File Index

### 3.1 File List

Here is a list of all files with brief descriptions:

|                                     |    |
|-------------------------------------|----|
| <a href="#">functions.cpp</a>       | 23 |
| <a href="#">functions.h</a>         | 27 |
| <a href="#">main.cpp</a>            | 34 |
| <a href="#">main.h</a>              | 35 |
| <a href="#">strategija.h</a>        | 38 |
| <a href="#">Strategija1.cpp</a>     | 45 |
| <a href="#">Strategija2.cpp</a>     | 46 |
| <a href="#">Strategija3.cpp</a>     | 46 |
| <a href="#">unitTesting.cpp</a>     | 46 |
| <a href="#">vector.h</a>            | 48 |
| <a href="#">vectorUnitTests.cpp</a> | 52 |



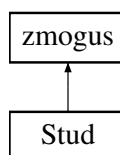
# Chapter 4

## Class Documentation

### 4.1 Stud Class Reference

```
#include <main.h>
```

Inheritance diagram for Stud:



#### Public Member Functions

- [Stud](#) ()
- [Stud](#) (const string &vrd, const string &pvr, int egzam, const vector< int > &namud)
- [Stud](#) (const [Stud](#) &studCopy)
- [Stud](#) & [operator=](#) (const [Stud](#) &studCopy)
- [Stud](#) ([Stud](#) &&studMove)
- [Stud](#) [operator=](#) ([Stud](#) &&studMove)
- void [clear](#) ()
- [~Stud](#) () override
- void [setVardas](#) (const string &var) override
- void [setPavarde](#) (const string &pav) override
- void [setEgzaminas](#) (const int &egz)
- void [setBalasGalutinisVid](#) (const double &Balas)
- void [setND](#) (int paz)
- int [getEgzaminas](#) () const
- int [getPaz](#) (int i) const
- const vector< int > & [getND](#) () const
- double [galutinisBalas](#) () const
- int [pazKiekis](#) () const
- const int & [grazintiPaskutini](#) () const
- void [removeLast](#) ()
- void [removeND](#) ()
- double [galutinis](#) () const
- bool [operator==](#) (const [Stud](#) &s) const
- bool [operator!=](#) (const [Stud](#) &s) const

## Public Member Functions inherited from [zmogus](#)

- [zmogus](#) ()
- [zmogus](#) (const string &var, const string &pav)
- virtual [~zmogus](#) ()
- string [getVardas](#) () const
- string [getPavarde](#) () const
- [zmogus](#) ([zmogus](#) &&z)
- [zmogus](#) & [operator=](#) ([zmogus](#) &&z)

## Friends

- std::ostream & [operator<<](#) (std::ostream &output, const [Stud](#) &studentas)
- std::istream & [operator>>](#) (std::istream &input, [Stud](#) &studentas)

## Additional Inherited Members

## Protected Attributes inherited from [zmogus](#)

- string [vardas](#)
- string [pavarde](#)

## 4.1.1 Constructor & Destructor Documentation

### 4.1.1.1 Stud() [1/4]

`Stud::Stud ()` [inline]

### 4.1.1.2 Stud() [2/4]

```
Stud::Stud (
    const string & vrđ,
    const string & pvrđ,
    int egzam,
    const vector< int > & namud) [inline]
```

### 4.1.1.3 Stud() [3/4]

```
Stud::Stud (
    const Stud & studCopy) [inline]
```

copy konstruktorius

### 4.1.1.4 Stud() [4/4]

```
Stud::Stud (
    Stud && studMove) [inline]
```

move konstruktorius



#### 4.1.1.5 ~Stud()

```
Stud::~~Stud () [inline], [override]
```

### 4.1.2 Member Function Documentation

#### 4.1.2.1 clear()

```
void Stud::clear () [inline]
```

#### 4.1.2.2 galutinis()

```
double Stud::galutinis () const [inline]
```

galutinio balo skaiciavimo metodas

#### 4.1.2.3 galutinisBalas()

```
double Stud::galutinisBalas () const [inline]
```

#### 4.1.2.4 getEgzaminas()

```
int Stud::getEgzaminas () const [inline]
```

#### 4.1.2.5 getND()

```
const vector< int > & Stud::getND () const [inline]
```

#### 4.1.2.6 getPaz()

```
int Stud::getPaz (
    int i) const [inline]
```

#### 4.1.2.7 grazintiPaskutini()

```
const int & Stud::grazintiPaskutini () const [inline]
```

#### 4.1.2.8 operator"!=()"

```
bool Stud::operator!= (
    const Stud & s) const [inline]
```

#### 4.1.2.9 operator=() [1/2]

```
Stud & Stud::operator= (
    const Stud & studCopy) [inline]
```

copy assignment operatorius

#### 4.1.2.10 operator=() [2/2]

```
Stud Stud::operator= (
    Stud && studMove) [inline]
```

move assignment operatorius

#### 4.1.2.11 operator==()

```
bool Stud::operator== (
    const Stud & s) const [inline]
```

#### 4.1.2.12 pazKiekis()

```
int Stud::pazKiekis () const [inline]
```

#### 4.1.2.13 removeLast()

```
void Stud::removeLast () [inline]
```

#### 4.1.2.14 removeND()

```
void Stud::removeND () [inline]
```

#### 4.1.2.15 setBalasGalutinisVid()

```
void Stud::setBalasGalutinisVid (
    const double & Balas) [inline]
```

#### 4.1.2.16 setEgzaminas()

```
void Stud::setEgzaminas (
    const int & egz) [inline]
```

#### 4.1.2.17 setND()

```
void Stud::setND (
    int paz) [inline]
```

#### 4.1.2.18 setPavarde()

```
void Stud::setPavarde (  
    const string & pav)    [inline], [override], [virtual]
```

Implements [zmogus](#).

#### 4.1.2.19 setVardas()

```
void Stud::setVardas (  
    const string & var)    [inline], [override], [virtual]
```

Implements [zmogus](#).

### 4.1.3 Friends And Related Symbol Documentation

#### 4.1.3.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & output,  
    const Stud & studentas)    [friend]
```

isvedimo operatorius

#### 4.1.3.2 operator>>

```
std::istream & operator>> (  
    std::istream & input,  
    Stud & studentas)    [friend]
```

inputo operatorius

The documentation for this class was generated from the following file:

- [main.h](#)

## 4.2 Vector< T > Class Template Reference

```
#include <vector.h>
```

## Public Member Functions

- [Vector](#) ()
- [Vector](#) (size\_t [Size](#))
- [Vector](#) (std::initializer\_list< T > list)
- [~Vector](#) ()
- [Vector](#) (const [Vector](#) &v)
- [Vector](#) & [operator=](#) (const [Vector](#) &v)
- [Vector](#) ([Vector](#) &&v)
- bool [operator==](#) (const [Vector](#) &vv) const
- bool [operator!=](#) (const [Vector](#) &vv) const
- [Vector](#) & [operator=](#) ([Vector](#) &&v) noexcept
- size\_t [Size](#) () const
- T \* [Data](#) () const
- size\_t [Capacity](#) () const
- size\_t [max\\_size](#) ()
- bool [empty](#) ()
- void [assign](#) (size\_t amount, T &value)
- void [assign](#) (T \*[begin](#), T \*[end](#))
- void [assign](#) (std::initializer\_list< T > list)
- void [append\\_range](#) (std::initializer\_list< T > list)
- T & [front](#) ()
- T & [back](#) ()
- T & [operator\[\]](#) (size\_t index)
- const T & [operator\[\]](#) (size\_t index) const
- T \* [end](#) ()
- T \* [begin](#) ()
- const T \* [cbegin](#) () const
- const T \* [cend](#) () const
- T \* [rbegin](#) ()
- T \* [rend](#) ()
- const T \* [crbegin](#) () const
- const T \* [crend](#) () const
- void [push\\_back](#) (const T &value)
- void [pop\\_back](#) ()
- T & [at](#) (size\_t index)
- void [clear](#) ()
- void [shrink\\_to\\_fit](#) ()
- void [swap](#) ([Vector](#) &v)
- void [reserve](#) (size\_t Ncap)
- void [insert](#) (T \*index, const T &element)
- void [erase](#) (T \*index)
- void [erase](#) (T \*[begin](#), T \*[end](#))
- void [resize](#) (size\_t newCap)
- void [resize](#) (size\_t newCap, const T &element)
- bool [operator<](#) (const [Vector](#) &v) const
- bool [operator<=](#) (const [Vector](#) &v) const
- bool [operator>](#) (const [Vector](#) &v) const
- bool [operator>=](#) (const [Vector](#) &v) const

## 4.2.1 Constructor & Destructor Documentation

### 4.2.1.1 [Vector](#)() [1/5]

```
template<typename T>
Vector< T >::Vector () [inline]
```

#### 4.2.1.2 Vector() [2/5]

```
template<typename T>
Vector< T >::Vector (
    size_t Size) [inline]
```

#### 4.2.1.3 Vector() [3/5]

```
template<typename T>
Vector< T >::Vector (
    std::initializer_list< T > list) [inline]
```

#### 4.2.1.4 ~Vector()

```
template<typename T>
Vector< T >::~~Vector () [inline]
```

#### 4.2.1.5 Vector() [4/5]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & v) [inline]
```

#### 4.2.1.6 Vector() [5/5]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && v) [inline]
```

### 4.2.2 Member Function Documentation

#### 4.2.2.1 append\_range()

```
template<typename T>
void Vector< T >::append_range (
    std::initializer_list< T > list) [inline]
```

#### 4.2.2.2 assign() [1/3]

```
template<typename T>
void Vector< T >::assign (
    size_t amount,
    T & value) [inline]
```

#### 4.2.2.3 assign() [2/3]

```
template<typename T>
void Vector< T >::assign (
    std::initializer_list< T > list) [inline]
```

#### 4.2.2.4 assign() [3/3]

```
template<typename T>
void Vector< T >::assign (
    T * begin,
    T * end) [inline]
```

#### 4.2.2.5 at()

```
template<typename T>
T & Vector< T >::at (
    size_t index) [inline]
```

#### 4.2.2.6 back()

```
template<typename T>
T & Vector< T >::back () [inline]
```

#### 4.2.2.7 begin()

```
template<typename T>
T * Vector< T >::begin () [inline]
```

#### 4.2.2.8 Capacity()

```
template<typename T>
size_t Vector< T >::Capacity () const [inline]
```

#### 4.2.2.9 cbegin()

```
template<typename T>
const T * Vector< T >::cbegin () const [inline]
```

#### 4.2.2.10 cend()

```
template<typename T>
const T * Vector< T >::cend () const [inline]
```

#### 4.2.2.11 clear()

```
template<typename T>
void Vector< T >::clear () [inline]
```

#### 4.2.2.12 crbegin()

```
template<typename T>
const T * Vector< T >::crbegin () const [inline]
```

#### 4.2.2.13 crend()

```
template<typename T>
const T * Vector< T >::crend () const [inline]
```

#### 4.2.2.14 Data()

```
template<typename T>
T * Vector< T >::Data () const [inline]
```

#### 4.2.2.15 empty()

```
template<typename T>
bool Vector< T >::empty () [inline]
```

#### 4.2.2.16 end()

```
template<typename T>
T * Vector< T >::end () [inline]
```

#### 4.2.2.17 erase() [1/2]

```
template<typename T>
void Vector< T >::erase (
    T * begin,
    T * end) [inline]
```

#### 4.2.2.18 erase() [2/2]

```
template<typename T>
void Vector< T >::erase (
    T * index) [inline]
```

#### 4.2.2.19 front()

```
template<typename T>
T & Vector< T >::front () [inline]
```

#### 4.2.2.20 insert()

```
template<typename T>
void Vector< T >::insert (
    T * index,
    const T & element) [inline]
```

#### 4.2.2.21 max\_size()

```
template<typename T>
size_t Vector< T >::max_size () [inline]
```

#### 4.2.2.22 operator"!="()

```
template<typename T>
bool Vector< T >::operator!= (
    const Vector< T > & vv) const [inline]
```

#### 4.2.2.23 operator<()

```
template<typename T>
bool Vector< T >::operator< (
    const Vector< T > & v) const [inline]
```

#### 4.2.2.24 operator<=()

```
template<typename T>
bool Vector< T >::operator<= (
    const Vector< T > & v) const [inline]
```

#### 4.2.2.25 operator=() [1/2]

```
template<typename T>
Vector & Vector< T >::operator= (
    const Vector< T > & v) [inline]
```

#### 4.2.2.26 operator=() [2/2]

```
template<typename T>
Vector & Vector< T >::operator= (
    Vector< T > && v) [inline], [noexcept]
```



#### 4.2.2.27 operator==( )

```
template<typename T>
bool Vector< T >::operator== (
    const Vector< T > & vv) const [inline]
```

#### 4.2.2.28 operator>( )

```
template<typename T>
bool Vector< T >::operator> (
    const Vector< T > & v) const [inline]
```

#### 4.2.2.29 operator>=( )

```
template<typename T>
bool Vector< T >::operator>= (
    const Vector< T > & v) const [inline]
```

#### 4.2.2.30 operator[]() [1/2]

```
template<typename T>
T & Vector< T >::operator[] (
    size_t index) [inline]
```

#### 4.2.2.31 operator[]() [2/2]

```
template<typename T>
const T & Vector< T >::operator[] (
    size_t index) const [inline]
```

#### 4.2.2.32 pop\_back()

```
template<typename T>
void Vector< T >::pop_back () [inline]
```

#### 4.2.2.33 push\_back()

```
template<typename T>
void Vector< T >::push_back (
    const T & value) [inline]
```

#### 4.2.2.34 rbegin()

```
template<typename T>
T * Vector< T >::rbegin () [inline]
```

#### 4.2.2.35 rend()

```
template<typename T>
T * Vector< T >::rend () [inline]
```

#### 4.2.2.36 reserve()

```
template<typename T>
void Vector< T >::reserve (
    size_t Ncap) [inline]
```

#### 4.2.2.37 resize() [1/2]

```
template<typename T>
void Vector< T >::resize (
    size_t newCap) [inline]
```

#### 4.2.2.38 resize() [2/2]

```
template<typename T>
void Vector< T >::resize (
    size_t newCap,
    const T & element) [inline]
```

#### 4.2.2.39 shrink\_to\_fit()

```
template<typename T>
void Vector< T >::shrink_to_fit () [inline]
```

#### 4.2.2.40 Size()

```
template<typename T>
size_t Vector< T >::Size () const [inline]
```

#### 4.2.2.41 swap()

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & v) [inline]
```

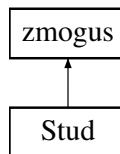
The documentation for this class was generated from the following file:

- [vector.h](#)

## 4.3 zmogus Class Reference

```
#include <main.h>
```

Inheritance diagram for zmogus:



### Public Member Functions

- [zmogus](#) ()
- [zmogus](#) (const string &var, const string &pav)
- virtual [~zmogus](#) ()
- virtual void [setVardas](#) (const string &var)=0
- virtual void [setPavarde](#) (const string &pav)=0
- string [getVardas](#) () const
- string [getPavarde](#) () const
- [zmogus](#) (zmogus &&z)
- [zmogus](#) & operator= (zmogus &&z)

### Protected Attributes

- string [vardas](#)
- string [pavarde](#)

## 4.3.1 Constructor & Destructor Documentation

### 4.3.1.1 zmogus() [1/3]

```
zmogus::zmogus () [inline]
```

### 4.3.1.2 zmogus() [2/3]

```
zmogus::zmogus (  
    const string & var,  
    const string & pav) [inline]
```

### 4.3.1.3 ~zmogus()

```
virtual zmogus::~~zmogus () [inline], [virtual]
```

#### 4.3.1.4 zmogus() [3/3]

```
zmogus::zmogus (  
    zmogus && z) [inline]
```

### 4.3.2 Member Function Documentation

#### 4.3.2.1 getPavarde()

```
string zmogus::getPavarde () const [inline]
```

#### 4.3.2.2 getVardas()

```
string zmogus::getVardas () const [inline]
```

#### 4.3.2.3 operator=()

```
zmogus & zmogus::operator= (  
    zmogus && z) [inline]
```

#### 4.3.2.4 setPavarde()

```
virtual void zmogus::setPavarde (  
    const string & pav) [pure virtual]
```

Implemented in [Stud](#).

#### 4.3.2.5 setVardas()

```
virtual void zmogus::setVardas (  
    const string & var) [pure virtual]
```

Implemented in [Stud](#).

### 4.3.3 Member Data Documentation

#### 4.3.3.1 pavarde

```
string zmogus::pavarde [protected]
```

#### 4.3.3.2 vardas

```
string zmogus::vardas [protected]
```

The documentation for this class was generated from the following file:

- [main.h](#)

# Chapter 5

## File Documentation

### 5.1 functions.cpp File Reference

```
#include "../functions.h"
```

#### Functions

- bool [tikrinimas](#) (const [Stud](#) &s, const [Stud](#) &s1)
- void [constrTest](#) ()
- void [copyConstrTest](#) ()
- void [copyAssignTest](#) ()
- void [moveConstrTest](#) ()
- void [moveAssignTest](#) ()
- std::ostream & [operator<<](#) (std::ostream &output, const [Stud](#) &studentas)
- std::istream & [operator>>](#) (std::istream &input, [Stud](#) &studentas)
- void [inputTest](#) ()
- void [outputTest](#) ()
- void [destructorTest](#) ()
- void [konstruktoriuTest](#) ()
- bool [tinkamas\\_char](#) (string vardas)
- bool [tinkamas\\_int](#) (int skaicius)
- void [pazymiu\\_ivedimas](#) ([Stud](#) &studentas, string &ss)
- void [paz\\_gener](#) ([Stud](#) &studentas, string &ss)
- [Stud](#) [vardai](#) ([Stud](#) &studentas, string &ss)
- void [duomenu\\_ivedimas](#) (vector< [Stud](#) > &studentai, int meniu)
- void [duomenu\\_generavimas](#) (vector< [Stud](#) > &studentai)
- string [filePasirinkimas](#) ()
- bool [PalygintiVardas](#) (const [Stud](#) &stud1, const [Stud](#) &stud2)
- bool [PalygintiPavardes](#) (const [Stud](#) &stud1, const [Stud](#) &stud2)
- bool [PalygintiKategorijas](#) (const [Stud](#) &stud1, const [Stud](#) &stud2)
- void [printCases](#) (vector< [Stud](#) > visi, bool outputFile, int RusiavimasPagal)
- void [print](#) (vector< [Stud](#) > visi, bool outputFile, int RusiavimasPagal)
- double [GeneruotiFiles](#) (int StudSkaicius)
- void [pushbackTest](#) ()

## 5.1.1 Function Documentation

### 5.1.1.1 constrTest()

```
void constrTest ()
```

### 5.1.1.2 copyAssignTest()

```
void copyAssignTest ()
```

### 5.1.1.3 copyConstrTest()

```
void copyConstrTest ()
```

### 5.1.1.4 destructorTest()

```
void destructorTest ()
```

### 5.1.1.5 duomenu\_generavimas()

```
void duomenu_generavimas (  
    vector< Stud > & studentai)
```

### 5.1.1.6 duomenu\_ivedimas()

```
void duomenu_ivedimas (  
    vector< Stud > & studentai,  
    int meniu)
```

### 5.1.1.7 filePasirinkimas()

```
string filePasirinkimas ()
```

### 5.1.1.8 GeneruotiFiles()

```
double GeneruotiFiles (  
    int StudSkaicius)
```

### 5.1.1.9 inputTest()

```
void inputTest ()
```

#### 5.1.1.10 konstruktoriuTest()

```
void konstruktoriuTest ()
```

#### 5.1.1.11 moveAssignTest()

```
void moveAssignTest ()
```

#### 5.1.1.12 moveConstTest()

```
void moveConstTest ()
```

#### 5.1.1.13 operator<<()

```
std::ostream & operator<< (  
    std::ostream & output,  
    const Stud & studentas)
```

isvedimo operatorius

#### 5.1.1.14 operator>>()

```
std::istream & operator>> (  
    std::istream & input,  
    Stud & studentas)
```

inputo operatorius

#### 5.1.1.15 outputTest()

```
void outputTest ()
```

#### 5.1.1.16 PalygintiKategorijas()

```
bool PalygintiKategorijas (  
    const Stud & stud1,  
    const Stud & stud2)
```

#### 5.1.1.17 PalygintiPavardes()

```
bool PalygintiPavardes (  
    const Stud & stud1,  
    const Stud & stud2)
```

#### 5.1.1.18 PalygintiVardas()

```
bool PalygintiVardas (  
    const Stud & stud1,  
    const Stud & stud2)
```

#### 5.1.1.19 paz\_gener()

```
void paz_gener (  
    Stud & studentas,  
    string & ss)
```

#### 5.1.1.20 pazymiu\_ivedimas()

```
void pazymiu_ivedimas (  
    Stud & studentas,  
    string & ss)
```

#### 5.1.1.21 print()

```
void print (  
    vector< Stud > visi,  
    bool outputFile,  
    int RusiavimasPagal)
```

#### 5.1.1.22 printCases()

```
void printCases (  
    vector< Stud > visi,  
    bool outputFile,  
    int RusiavimasPagal)
```

#### 5.1.1.23 pushbackTest()

```
void pushbackTest ()
```

#### 5.1.1.24 tikrinimas()

```
bool tikrinimas (  
    const Stud & s,  
    const Stud & s1)
```

#### 5.1.1.25 tinkamas\_char()

```
bool tinkamas_char (  
    string vardas)
```



### 5.1.1.26 tinkamas\_int()

```
bool tinkamas_int (
    int skaicius)
```

### 5.1.1.27 vardai()

```
Stud vardai (
    Stud & studentas,
    string & ss)
```

## 5.2 functions.h File Reference

```
#include "main.h"
#include "vectorClass/vector.h"
```

### Functions

- bool [tinkamas\\_char](#) (string vardas)
- bool [tinkamas\\_int](#) (int skaicius)
- void [pazymiu\\_ivedimas](#) (Stud &studentas, string &ss)
- void [paz\\_gener](#) (Stud &studentas, string &ss)
- Stud [vardai](#) (Stud &studentas, string &ss)
- void [duomenu\\_ivedimas](#) (vector< Stud > &studentai, int meniu)
- void [duomenu\\_generavimas](#) (vector< Stud > &studentai)
- void [printCases](#) (vector< Stud > visi, bool outputFile, int RusiavimasPagal)
- void [konstruktoriuTest](#) ()
- string [filePasirinkimas](#) ()
- void [pushbackTest](#) ()
- bool [tikrinimas](#) (const Stud &s, const Stud &s1)
- template<typename konteineris>  
void [fileskait](#) (konteineris &studentai, bool a, string filePav, double &BendrasLaikas)
- double [GeneruotiFiles](#) (int StudSkaicius)
- bool [PalygintiVardas](#) (const Stud &stud1, const Stud &stud2)
- bool [PalygintiPavardes](#) (const Stud &stud1, const Stud &stud2)
- bool [PalygintiKategorijas](#) (const Stud &stud1, const Stud &stud2)
- template<typename konteineris>  
void [PrintVektorius](#) (konteineris &nesimokantys, konteineris &pirmunai, int a, int RusiavimasPagal, double &BendrasLaikas)
- template<typename konteineris>  
void [vectorIdejimas](#) (int studSkaicius, konteineris &pirmunai, konteineris &nesimokantys, double &BendrasLaikas, konteineris &studentaiTest)
- template<typename konteineris>  
void [tyrimai](#) (int pasirinkimasTyrimo, konteineris &pirmunai, konteineris &nesimokantys, konteineris &studentaiTest)

## 5.2.1 Function Documentation

### 5.2.1.1 duomenu\_generavimas()

```
void duomenu_generavimas (  
    vector< Stud > & studentai)
```

### 5.2.1.2 duomenu\_ivedimas()

```
void duomenu_ivedimas (  
    vector< Stud > & studentai,  
    int menu)
```

### 5.2.1.3 filePasirinkimas()

```
string filePasirinkimas ()
```

### 5.2.1.4 fileskait()

```
template<typename konteineris>  
void fileskait (  
    konteineris & studentai,  
    bool a,  
    string filePav,  
    double & BendrasLaikas)
```

### 5.2.1.5 GeneruotiFiles()

```
double GeneruotiFiles (  
    int StudSkaicius)
```

### 5.2.1.6 konstruktoriuTest()

```
void konstruktoriuTest ()
```

### 5.2.1.7 PalygintiKategorijas()

```
bool PalygintiKategorijas (  
    const Stud & stud1,  
    const Stud & stud2)
```

### 5.2.1.8 PalygintiPavardes()

```
bool PalygintiPavardes (  
    const Stud & stud1,  
    const Stud & stud2)
```

### 5.2.1.9 PalygintiVardas()

```
bool PalygintiVardas (  
    const Stud & stud1,  
    const Stud & stud2)
```

### 5.2.1.10 paz\_gener()

```
void paz_gener (  
    Stud & studentas,  
    string & ss)
```

### 5.2.1.11 pazymiu\_ivedimas()

```
void pazymiu_ivedimas (  
    Stud & studentas,  
    string & ss)
```

### 5.2.1.12 printCases()

```
void printCases (  
    vector< Stud > visi,  
    bool outputFILE,  
    int RusiavimasPagal)
```

### 5.2.1.13 PrintVektorius()

```
template<typename konteineris>  
void PrintVektorius (  
    konteineris & nesimokantys,  
    konteineris & pirmunai,  
    int a,  
    int RusiavimasPagal,  
    double & BendrasLaikas)
```

### 5.2.1.14 pushbackTest()

```
void pushbackTest ()
```

### 5.2.1.15 tikrinimas()

```
bool tikrinimas (  
    const Stud & s,  
    const Stud & s1)
```

### 5.2.1.16 tinkamas\_char()

```
bool tinkamas_char (
    string vardas)
```

### 5.2.1.17 tinkamas\_int()

```
bool tinkamas_int (
    int skaicius)
```

### 5.2.1.18 tyrimai()

```
template<typename konteineris>
void tyrimai (
    int pasirinkimasTyrimo,
    konteineris & pirmunai,
    konteineris & nesimokantys,
    konteineris & studentaiTest)
```

### 5.2.1.19 vardai()

```
Stud vardai (
    Stud & studentas,
    string & ss)
```

### 5.2.1.20 vectorIdejimas()

```
template<typename konteineris>
void vectorIdejimas (
    int studSkaicius,
    konteineris & pirmunai,
    konteineris & nesimokantys,
    double & BendrasLaikas,
    konteineris & studentaiTest)
```

## 5.3 functions.h

[Go to the documentation of this file.](#)

```
00001
00002 #ifndef FUNCTIONS_H
00003 #define FUNCTIONS_H
00004 #include "main.h"
00005 #include "vectorClass/vector.h"
00006
00007 bool tinkamas_char(string vardas);
00008 bool tinkamas_int(int skaicius);
00009 //double galutinis(Stud studentas, int PagalSkaiciavimas);
00010 void pazymiu_ivedimas(Stud &studentas, string& ss);
00011 void paz_gener(Stud &studentas, string& ss);
00012 Stud vardai(Stud &studentas, string& ss);
00013 void duomenu_ivedimas(vector<Stud> &studentai, int meniu);
00014 void duomenu_generavimas(vector<Stud> &studentai);
00015 void printCases(vector<Stud> visi, bool outputFILE, int RusiavimasPagal);
```

```

00016 void konstruktoriuTest();
00017 string filePasirinkimas();
00018 void pushbackTest();
00019 bool tikrinimas(const Stud& s, const Stud& s1);
00020
00021 template <typename konteineris>
00022 void fileskait(konteineris &studentai, bool a, string filePav, double &BendrasLaikas)
00023 {
00024     double visasLaikas = 0.0;
00025     vector<string> visaeil;
00026     int pazymys;
00027     string eilute;
00028     string eilute1;
00029     ifstream f;
00030     std::stringstream bufferis;
00031     string testuojamasFile;
00032     if (a == false){
00033         testuojamasFile = filePasirinkimas();
00034     }
00035     if (a == true){
00036         testuojamasFile = filePav;
00037     }
00038     cout << "...\\n";
00039     cout << testuojamasFile << endl;
00040
00041     if (testuojamasFile != "kursiokai.txt"){
00042         testuojamasFile = "Strategijos\\Tyrimo_files\\" + testuojamasFile;
00043     }
00044
00045     while (true)
00046     {
00047         try
00048         {
00049             f.open(testuojamasFile);
00050             if (!f)
00051             {
00052                 throw "Failo atidaryti nepavyko";
00053             }
00054             break;
00055         }
00056         catch (const char *masyvas)
00057         {
00058             cout << masyvas << endl;
00059             testuojamasFile = filePasirinkimas();
00060
00061             continue;
00062         }
00063     }
00064     int iteracijos;
00065     (testuojamasFile == "kursiokai.txt") ? iteracijos = 0 : iteracijos = 2;
00066     for (int i = 0; i <= iteracijos; i++)
00067     {
00068         studentai.clear();
00069         bufferis.str("");
00070         bufferis.clear();
00071         auto start = std::chrono::high_resolution_clock::now();
00072         bufferis << f.rdbuf();
00073         f.close();
00074         getline(bufferis, eilute1);
00075         while (bufferis)
00076         {
00077             if (!bufferis.eof())
00078             {
00079                 getline(bufferis, eilute);
00080                 visaeil.push_back(eilute);
00081             }
00082             else
00083                 break;
00084         }
00085         for (string& s : visaeil){
00086             Stud studentas;
00087             std::istringstream f(s);
00088             f >> studentas;
00089             studentai.push_back(studentas);
00090         }
00091         auto end = std::chrono::high_resolution_clock::now();
00092         std::chrono::duration<double> diff = end - start;
00093         if (i != 0){
00094             visasLaikas += diff.count();
00095         }
00096         if (testuojamasFile != "kursiokai.txt" && a == false)
00097         {
00098             cout << i + 1 << " iteracija: " << std::fixed << std::setprecision(3) << diff.count() << "s" << endl;
00099         }
00100     }
00101     if (testuojamasFile != "kursiokai.txt")
00102     {

```

```

00103
00104     if (a == false)
00105     {
00106         cout << "Programos vidutinis vykdymo laikas: " << std::fixed << std::setprecision(3) << visasLaikas / 2 <<
"s" << endl;
00107     }
00108     if (a == true)
00109     {
00110         cout << "Nuskaitymas is failo vidutiniskai truko: " << std::fixed << std::setprecision(3) << visasLaikas / 2
<< "s" << endl;
00111         BendrasLaikas += (visasLaikas / 2);
00112     }
00113 }
00114 }
00115
00116 double GeneruotiFiles(int StudSkaicius);
00117 bool PalygintiVardas(const Stud &stud1, const Stud &stud2);
00118 bool PalygintiPavardes(const Stud &stud1, const Stud &stud2);
00119 bool PalygintiKategorijas(const Stud &stud1, const Stud &stud2);
00120
00121 template <typename konteineris>
00122 void PrintVektorius(konteineris &nesimokantys, konteineris &pirmunai, int a, int RusiavimasPagal, double
&BendrasLaikas)
00123 {
00124     string FILEMOK = "Pirmunai" + std::to_string(a) + ".txt";
00125     string FILENESIMOK = "Nesimokantys" + std::to_string(a) + ".txt";
00126     std::stringstream buferis;
00127     std::ofstream f, F;
00128     f.open(FILEMOK);
00129
00130     if (RusiavimasPagal == 1)
00131     {
00132         double LaikasRusiavimo = 0.0;
00133         for (int t = 0; t < 3; t++){
00134             {
00135                 auto t1 = std::chrono::high_resolution_clock::now();
00136                 if constexpr (std::is_same_v<konteineris, vector<Stud> || std::is_same_v<konteineris, deque<Stud>){
00137                     sort(nesimokantys.begin(), nesimokantys.end(), PalygintiVardas);
00138                     sort(pirmunai.begin(), pirmunai.end(), PalygintiVardas);}
00139                 else{
00140                     nesimokantys.sort(PalygintiVardas);
00141                     pirmunai.sort(PalygintiVardas);}
00142
00143                 auto t2 = std::chrono::high_resolution_clock::now();
00144                 std::chrono::duration<double> diff = t2 - t1;
00145                 if (t != 0){
00146                     LaikasRusiavimo += diff.count();}
00147             }
00148
00149             cout << "Rusiavimas vidutiniskai uztruko: " << LaikasRusiavimo / 2 << "s" << endl;
00150             BendrasLaikas += (LaikasRusiavimo / 2);
00151         }
00152     else if (RusiavimasPagal == 2){
00153         double LaikasRusiavimo = 0.0;
00154         for (int t = 0; t < 3; t++){
00155             auto t1 = std::chrono::high_resolution_clock::now();
00156             if constexpr (std::is_same_v<konteineris, vector<Stud> || std::is_same_v<konteineris, deque<Stud>){
00157                 sort(nesimokantys.begin(), nesimokantys.end(), PalygintiPavardes);
00158                 sort(pirmunai.begin(), pirmunai.end(), PalygintiPavardes);
00159             }
00160             else{
00161                 nesimokantys.sort(PalygintiPavardes);
00162                 pirmunai.sort(PalygintiPavardes);
00163             }
00164             auto t2 = std::chrono::high_resolution_clock::now();
00165             std::chrono::duration<double> diff = t2 - t1;
00166             if (t != 0){
00167                 LaikasRusiavimo += diff.count();}
00168             }
00169             cout << "Rusiavimas vidutiniskai uztruko: " << LaikasRusiavimo / 2 << "s" << endl;
00170             BendrasLaikas += (LaikasRusiavimo / 2);
00171         }
00172     else if (RusiavimasPagal == 3){
00173         double LaikasRusiavimo = 0.0;
00174         for (int t = 0; t < 3; t++){
00175             auto t1 = std::chrono::high_resolution_clock::now();
00176             if constexpr (std::is_same_v<konteineris, vector<Stud> || std::is_same_v<konteineris, deque<Stud>){
00177                 sort(nesimokantys.begin(), nesimokantys.end(), PalygintiKategorijas);
00178                 sort(pirmunai.begin(), pirmunai.end(), PalygintiKategorijas);
00179             }
00180             else
00181             {
00182                 nesimokantys.sort(PalygintiKategorijas);
00183                 pirmunai.sort(PalygintiKategorijas);
00184             }
00185             auto t2 = std::chrono::high_resolution_clock::now();
00186             std::chrono::duration<double> diff = t2 - t1;

```

```

00187         if (t != 0){
00188             LaikasRusiavimo += difft.count();
00189         }
00190     }
00191     cout << "Rusiavimas vidutiniskai uztruko: " << LaikasRusiavimo / 2 << "s" << endl;
00192     BendrasLaikas += (LaikasRusiavimo / 2);
00193 }
00194 else if (RusiavimasPagal == 4){
00195     cout << "Rusiavimas uztruko: 0.000 s" << endl;
00196 }
00197
00198 double IsvedimoLaikas = 0.0;
00199 for (int k = 0; k < 2; k++){
00200     auto start2 = std::chrono::high_resolution_clock::now();
00201     buferis << std::setw(16) << std::left << "Pavarde" << std::setw(16) << std::left << "Vardas" << std::setw(16) <<
std::left << "Galutinis (Vid.)\n";
00202     buferis << "-----" << endl;
00203     for (Stud j : pirmunai)
00204     {
00205         buferis << std::setw(16) << std::left << j.getPavarde() << std::setw(16) << std::left << j.getVardas() <<
std::setw(16) << std::fixed << std::setprecision(2) << j.galutinisBalas() << endl;
00206     }
00207     f << buferis.rdbuf();
00208     buferis.str("");
00209     buferis.clear();
00210     f.close();
00211     pirmunai.clear();
00212     f.open(FILENAMESIMOK);
00213     buferis << std::setw(16) << std::left << "Pavarde" << std::setw(16) << std::left << "Vardas" << std::setw(16) <<
std::left << "Galutinis (Vid.)\n";
00214     buferis << "-----" << endl;
00215     for (Stud j : nesimokantys)
00216     {
00217         buferis << std::setw(16) << std::left << j.getPavarde() << std::setw(16) << std::left << j.getVardas() <<
std::setw(16) << std::fixed << std::setprecision(2) << j.galutinisBalas() << endl;
00218     }
00219     f << buferis.rdbuf();
00220     f.close();
00221     auto end2 = std::chrono::high_resolution_clock::now();
00222     std::chrono::duration<double> diff2 = end2 - start2;
00223     }
00224     nesimokantys.clear();
00225 }
00226
00227 template <typename konteineris>
00228 void vectorIdejimas(int studSkaicius, konteineris &pirmunai, konteineris &nesimokantys, double &BendrasLaikas,
konteineris &studentaiTest)
00229 {
00230     string file = "Studentai" + std::to_string(studSkaicius) + ".txt";
00231     studentaiTest;
00232     fileskait(studentaiTest, true, file, BendrasLaikas);
00233     double visasLaikas1 = 0.0;
00234     for (int c = 0; c < 2; c++)
00235     {
00236         auto start1 = std::chrono::high_resolution_clock::now();
00237         for (Stud& j : studentaiTest)
00238         {
00239             if (j.galutinisBalas() >= 5)
00240             {
00241                 pirmunai.push_back(j);
00242             }
00243             else
00244                 nesimokantys.push_back(j);
00245         }
00246         auto end1 = std::chrono::high_resolution_clock::now();
00247         studentaiTest.clear();
00248         std::chrono::duration<double> diff1 = end1 - start1;
00249         visasLaikas1 += diff1.count();
00250     }
00251     if constexpr (std::is_same_v<konteineris, vector<Stud> || std::is_same_v<konteineris, deque<Stud>)
00252     {
00253         pirmunai.shrink_to_fit();
00254         nesimokantys.shrink_to_fit();
00255     }
00256
00257     cout << "Duomeni isskirstymas i dvi grupes vidutiniskai trunka: " << visasLaikas1 / 2 << "s\n";
00258     BendrasLaikas += visasLaikas1 / 2;
00259 }
00260
00261 template <typename konteineris>
00262 void tyrimai(int pasirinkimasTyrimo, konteineris &pirmunai, konteineris &nesimokantys, konteineris &studentaiTest)
00263 {
00264     if (pasirinkimasTyrimo == 1)
00265     {
00266         for (int a = 1000; a <= 10000000; a *= 10)
00267         {
00268             cout << "Studentai" << a << ".txt\n";

```

```

00269         double laikas = 0.0;
00270         for (int i = 0; i < 2; i++)
00271         {
00272             double s = GeneruotiFiles(a);
00273             cout < i + 1 < " iteracija: " < s < "s" < endl;
00274             laikas += s;
00275         }
00276         cout < "Studentai" < a < ".txt vidutiniskai generuoja: " < laikas / 2 < "s" < endl;
00277         cout < "...\\n";
00278     }
00279 }
00280 else if (pasirinkimasTyrimo == 2)
00281 {
00282     int RusiavimasPagal;
00283     cout < "Studentus rusiuoti pagal: 1 - vardus, 2 - pavardes, 3 - galutini bala, 4 - duomeni nerusiuoti\\n";
00284     while (true){
00285         try
00286         {
00287             cin > RusiavimasPagal;
00288             if (cin.fail())
00289             {
00290                 cin.clear();
00291                 cin.ignore(numeric_limits<streamsize>::max(), '\\n');
00292                 throw "Neteisingas pasirinkimas, iveskite skaiciu 1-4";
00293                 break;
00294             }
00295             else if (RusiavimasPagal > 4 || RusiavimasPagal < 1){
00296                 throw "Neteisingas pasirinkimas, iveskite skaiciu 1-4";
00297                 break;
00298             }
00299             break;
00300         }
00301         catch (const char *masyvas){
00302             cout < masyvas < endl;}
00303     }
00304     for (int a = 1000; a <= 10000000; a *= 10)
00305     {
00306         double BendrasVidLaikas = 0.0;
00307         vectorIdejimas(a, pirmunai, nesimokantys, BendrasVidLaikas, studentaiTest);
00308         PrintVektorius(nesimokantys, pirmunai, a, RusiavimasPagal, BendrasVidLaikas);
00309         cout < "Visos programos vykdymo laikas: ";
00310         cout < BendrasVidLaikas < "s" < endl;
00311         cout < endl;
00312     }
00313 }
00314 }
00315 #endif

```

## 5.4 main.cpp File Reference

```
#include "functions.h"
```

### Functions

- int [main](#) ()

### 5.4.1 Function Documentation

#### 5.4.1.1 main()

```
int main ()
```



## 5.5 main.h File Reference

```
#include <iostream>
#include <iomanip>
#include <stdio.h>
#include <stdlib.h>
#include <fstream>
#include <sstream>
#include <vector>
#include <chrono>
#include <cpuid.h>
#include <string>
#include <ctime>
#include <cmath>
#include <algorithm>
#include <cctype>
#include <filesystem>
#include <random>
#include <deque>
#include <list>
#include <execution>
#include <thread>
```

### Classes

- class [zmogus](#)
- class [Stud](#)

### Typedefs

- using [konteinerisVector](#) = std::vector<[Stud](#)>
- using [konteinerisList](#) = std::list<[Stud](#)>
- using [konteinerisDeque](#) = std::deque<[Stud](#)>

## 5.5.1 Typedef Documentation

### 5.5.1.1 konteinerisDeque

```
using konteinerisDeque = std::deque<Stud>
```

### 5.5.1.2 konteinerisList

```
using konteinerisList = std::list<Stud>
```

### 5.5.1.3 konteinerisVector

```
using konteinerisVector = std::vector<Stud>
```

## 5.6 main.h

[Go to the documentation of this file.](#)

```

00001 #ifndef main_h
00002 #define main_h
00003
00004 #include <iostream>
00005 #include <iomanip>
00006 #include <stdio.h>
00007 #include <stdlib.h>
00008 #include <fstream>
00009 #include <sstream>
00010 #include <vector>
00011 #include <chrono>
00012 #include <cpuid.h>
00013 #include <string>
00014 #include <ctime>
00015 #include <cmath>
00016 #include <algorithm>
00017 #include <cctype>
00018 #include <filesystem>
00019 #include <random>
00020 #include <filesystem>
00021 #include <deque>
00022 #include <list>
00023 #include <execution>
00024 #include <thread>
00025
00026 using std::cin;
00027 using std::cout;
00028 using std::endl;
00029 using std::getline;
00030 using std::ifstream;
00031 using std::ios;
00032 using std::isalpha;
00033 using std::isdigit;
00034 using std::isspace;
00035 using std::numeric_limits;
00036 using std::sort;
00037 using std::streamsize;
00038 using std::string;
00039 using std::vector;
00040 using std::deque;
00041 using std::list;
00042
00043 class zmogus{
00044     protected:
00045         string vardas;
00046         string pavarde;
00047     public:
00048         zmogus() : vardas(""), pavarde("") {}
00049         zmogus(const string& var, const string& pav) : vardas(var), pavarde(pav) {}
00050         virtual ~zmogus() { vardas = ""; pavarde="";}
00051
00052         virtual void setVardas(const string& var) = 0;
00053         virtual void setPavarde(const string& pav) = 0;
00054
00055         string getVardas()const{return vardas;}
00056         string getPavarde()const{return pavarde;}
00057
00058         //Move constructor
00059         zmogus(zmogus&& z): vardas(std::move(z.vardas)), pavarde(std::move(z.pavarde)) {}
00060
00061         //Move assignment operator
00062         zmogus& operator=(zmogus&& z) {
00063             if (this != &z) {
00064                 vardas = std::move(z.vardas);
00065                 pavarde = std::move(z.pavarde);}
00066             return *this;}
00067 };
00068
00069 class Stud : public zmogus{
00070     private:
00071         //string vardas, pavarde;
00072         int egzaminas;
00073         vector<int> nd;
00074         mutable double BalasGalutinisVid;
00075         mutable bool balasSuskaiciuotas;
00076     public:
00077         Stud() : zmogus("", ""), egzaminas(0), nd{}, BalasGalutinisVid(0), balasSuskaiciuotas(false) {};
00078         //konstruktorius defaultinis
00079         Stud(const string& vrd, const string& pvrđ, int egzam, const vector<int>& namud): zmogus(vrd, pvrđ),
00080             egzaminas(egzam), nd(namud), BalasGalutinisVid(0), balasSuskaiciuotas(false) {}
00081
00082         Stud(const Stud& studCopy) :
```

```

00083     zmogus(studCopy.getVardas(), studCopy.getPavarde()),
00084     egzaminas(studCopy.egzaminas),
00085     nd(studCopy.nd),
00086     BalasGalutinisVid(studCopy.BalasGalutinisVid),
00087     balasSuskaiciuotas(studCopy.balasSuskaiciuotas) {}
00088
00089
00090
00092 Stud& operator=(const Stud& studCopy){
00093     if(this != &studCopy){
00094         setVardas(studCopy.getVardas());
00095         setPavarde(studCopy.getPavarde());
00096         nd = studCopy.nd;
00097         egzaminas = studCopy.egzaminas;
00098         BalasGalutinisVid = studCopy.BalasGalutinisVid;
00099         balasSuskaiciuotas = studCopy.balasSuskaiciuotas;}
00100     return *this;
00101 }
00102
00104 Stud(Stud&& studMove) :
00105     zmogus(std::move(studMove)),
00106     nd(std::move(studMove.nd)),
00107     egzaminas(std::move(studMove.egzaminas)),
00108     BalasGalutinisVid(std::move(studMove.BalasGalutinisVid)),
00109     balasSuskaiciuotas(std::move(studMove.balasSuskaiciuotas)){studMove.clear();}
00110
00112 Stud operator=(Stud &&studMove)
00113 {
00114     if (this != &studMove)
00115     {
00116         zmogus::operator=(std::move(studMove));
00117         nd = std::move(studMove.nd);
00118         egzaminas = std::move(studMove.egzaminas);
00119         BalasGalutinisVid = std::move(studMove.BalasGalutinisVid);
00120         balasSuskaiciuotas = std::move(studMove.balasSuskaiciuotas);
00121         studMove.clear();
00122     }
00123     return *this;
00124 }
00125
00126
00127 void clear(){nd.clear(); egzaminas=0; BalasGalutinisVid=0;balasSuskaiciuotas=false;}
00128 ~Stud()override{clear();} //destruktorius
00129
00130 //setteriai
00131 void setVardas(const string& var)override{vardas=var;}
00132 void setPavarde(const string& pav)override{pavarde=pav;}
00133 void setEgzaminas(const int& egz){egzaminas=egz;}
00134 void setBalasGalutinisVid(const double& Balas){BalasGalutinisVid=Balas; balasSuskaiciuotas = true;}
00135 void setND(int paz){nd.push_back(paz);}
00136
00137 //getteriai
00138 int getEgzaminas() const {return egzaminas;}
00139 int getPaz(int i) const {return nd[i];}
00140 const vector<int>& getND() const {return nd;}
00141 double galutinisBalas() const {return BalasGalutinisVid;}
00142 int pazKiekis() const {return nd.size();}
00143 const int& grazintiPaskutini() const {return nd.back();}
00144 void removeLast(){nd.pop_back();}
00145 void removeND(){nd.clear();}
00146
00148 double galutinis() const{
00149     if(!balasSuskaiciuotas){
00150         double suma = 0;
00151         double vidurkis;
00152         if (nd.size() == 1) {
00153             vidurkis = nd[0];}
00154         else {
00155             for (size_t ss = 0; ss < nd.size(); ss++) {
00156                 suma += nd[ss];}
00157             vidurkis = suma / nd.size();}
00158         BalasGalutinisVid = round((0.4 * vidurkis + 0.6 * egzaminas) * 100) / 100;
00159         balasSuskaiciuotas = true;
00160     }
00161     return BalasGalutinisVid;
00162 }
00163 //lygybes operatorius testavimui
00164 bool operator==(const Stud &s) const{
00165     if(getVardas()==s.getVardas() && getPavarde()==s.getPavarde() && getEgzaminas()==s.getEgzaminas()
00166     && getND()== s.getND()){return true;}
00167     else return false;
00168 }
00169 //nelygybes operatorius testavimui
00170 bool operator!=(const Stud &s)const{
00171     if(getVardas() != s.getVardas() || getPavarde() != s.getPavarde() || getEgzaminas() != s.getEgzaminas()
00172     || getND() != s.getND()){return true;}
00173     else return false;

```

```

00174     }
00175
00176     friend std::ostream& operator<(std::ostream &output, const Stud& studentas);
00177     friend std::istream& operator>(std::istream &input, Stud& studentas);
00178 };
00179
00180 using konteinerisVector = std::vector<Stud>;
00181 using konteinerisList = std::list<Stud>;
00182 using konteinerisDeque = std::deque<Stud>;
00183
00184 #endif

```

## 5.7 strategija.h File Reference

```

#include "../functions.h"
#include "../vectorClass/vector.h"

```

### Functions

- void [nuskaitymasFile](#) (string filePavadinimas, double &visasLaikas, [Vector](#)< [Stud](#) > &studentai)
- template<typename konteineris>  
void [studentuIsskirstymas](#) (konteineris &studentai, double &visasLaikas, konteineris &pirmunai, konteineris &nesimokantys)
- template<typename konteineris>  
void [studentuRusiavimas](#) (int &rusiavimasPagal, konteineris &pirmunai, konteineris &nesimokantys, double &visasLaikas)
- template<typename konteineris>  
void [testavimasPrint](#) (konteineris &studentai, konteineris &pirmunai, konteineris &nesimokantys, int a)
- template<typename konteineris>  
void [vykdomaPrograma](#) (int &rusiavimasPagal, konteineris &studentai, konteineris &pirmunai, konteineris &nesimokantys)
- template<typename konteineris>  
void [studentuRusiavimas2](#) (int rusiavimasPagal, konteineris &studentai, double &visasLaikas, double &rusiavimoLaikas)
- template<typename konteineris>  
void [studentuIsskirstymas2](#) (konteineris &studentai, double &visasLaikas, konteineris &nesimokantys, double &rusiavimoLaikas)
- template<typename konteineris>  
void [vykdomaPrograma2](#) (int rusiavimasPagal, konteineris &studentai, konteineris &nesimokantys)
- void [studentuSkirstymas3](#) ([Vector](#)< [Stud](#) > &studentai, [Vector](#)< [Stud](#) > &nesimokantys, double &visasLaikas)
- void [StudentuRusiavimas3](#) ([Vector](#)< [Stud](#) > &Studentai, [Vector](#)< [Stud](#) > &nesimokantys, double &visasLaikas, int RusiuotiPagal)
- void [vykdomaPrograma3](#) (int rusiavimasPagal, [Vector](#)< [Stud](#) > &studentai, [Vector](#)< [Stud](#) > &nesimokantys)

### 5.7.1 Function Documentation

#### 5.7.1.1 nuskaitymasFile()

```

void nuskaitymasFile (
    string filePavadinimas,
    double & visasLaikas,
    Vector< Stud > & studentai)

```

### 5.7.1.2 studentuIsskirstymas()

```
template<typename konteineris>
void studentuIsskirstymas (
    konteineris & studentai,
    double & visasLaikas,
    konteineris & pirmunai,
    konteineris & nesimokantys)
```

### 5.7.1.3 studentuIsskirstymas2()

```
template<typename konteineris>
void studentuIsskirstymas2 (
    konteineris & studentai,
    double & visasLaikas,
    konteineris & nesimokantys,
    double & rusiavimoLaikas)
```

### 5.7.1.4 studentuRusiavimas()

```
template<typename konteineris>
void studentuRusiavimas (
    int & rusiavimasPagal,
    konteineris & pirmunai,
    konteineris & nesimokantys,
    double & visasLaikas)
```

### 5.7.1.5 studentuRusiavimas2()

```
template<typename konteineris>
void studentuRusiavimas2 (
    int rusiavimasPagal,
    konteineris & studentai,
    double & visasLaikas,
    double & rusiavimoLaikas)
```

### 5.7.1.6 StudentuRusiavimas3()

```
void StudentuRusiavimas3 (
    Vector< Stud > & Studentai,
    Vector< Stud > & nesimokantys,
    double & visasLaikas,
    int RusiuotiPagal)
```

### 5.7.1.7 studentuSkirstymas3()

```
void studentuSkirstymas3 (
    Vector< Stud > & studentai,
    Vector< Stud > & nesimokantys,
    double & visasLaikas)
```

### 5.7.1.8 testavimasPrint()

```
template<typename konteineris>
void testavimasPrint (
    konteineris & studentai,
    konteineris & pirmunai,
    konteineris & nesimokantys,
    int a)
```

### 5.7.1.9 vykdomaPrograma()

```
template<typename konteineris>
void vykdomaPrograma (
    int & rusiavimasPagal,
    konteineris & studentai,
    konteineris & pirmunai,
    konteineris & nesimokantys)
```

### 5.7.1.10 vykdomaPrograma2()

```
template<typename konteineris>
void vykdomaPrograma2 (
    int rusiavimasPagal,
    konteineris & studentai,
    konteineris & nesimokantys)
```

### 5.7.1.11 vykdomaPrograma3()

```
void vykdomaPrograma3 (
    int rusiavimasPagal,
    Vector< Stud > & studentai,
    Vector< Stud > & nesimokantys)
```

## 5.8 strategija.h

[Go to the documentation of this file.](#)

```
00001 #ifndef strategija_h
00002 #define strategija_h
00003 #include "../functions.h"
00004 #include "../vectorClass/vector.h"
00005
00006 //template <typename konteineris>
00007 void nuskaitymasFile(string filePavadinimas, double &visasLaikas, Vector<Stud> &studentai)//cia pakeisti
00008 {
00009     double laikas = 0.0;
00010     Vector<string> visaeil;
00011     int pazymys;
00012     string eilute;
00013     string eilute1;
00014     ifstream f;
00015     std::stringstream bufferis;
00016     f.open(filePavadinimas);
00017     for (int i = 0; i < 3; i++)
00018     {
00019         studentai.clear();
00020         bufferis.clear();
00021         bufferis.str("");
```

```

00022     auto start = std::chrono::high_resolution_clock::now();
00023     bufferis << f.rdbuf();
00024     f.close();
00025     getline(bufferis, eilute1);
00026     while (bufferis)
00027     {
00028         if (!bufferis.eof())
00029         {
00030             getline(bufferis, eilute);
00031             visaeil.push_back(eilute);
00032         }
00033         else
00034             break;
00035     }
00036
00037     for (const string& s : visaeil)
00038     {
00039         string var, pav;
00040         Stud studentas;
00041         std::istream f(s);
00042         f >> var >> pav;
00043         studentas.setVardas(var);
00044         studentas.setPavarde(pav);
00045         while (f >> pazymys)
00046         {
00047             studentas.setND(pazymys);
00048         }
00049         studentas.setEgzaminas(studentas.grazintiPaskutini());
00050         studentas.removeLast();
00051         studentas.galutinis();
00052         studentai.push_back(studentas);
00053     }
00054
00055     auto end = std::chrono::high_resolution_clock::now();
00056     std::chrono::duration<double> diff = end - start;
00057     if (i != 0)
00058     {
00059         laikas += diff.count();
00060     }
00061 }
00062
00063 visasLaikas += (laikas / 2);
00064 cout << "Failo nuskaitymas vidutiniskai truko: " << laikas / 2 << "s" << endl;
00065 studentai.shrink_to_fit();
00066 visaeil.clear();
00067 /*if constexpr (std::is_same_v<konteineris, vector<Stud> || std::is_same_v<konteineris, deque<Stud>)
00068 {
00069     studentai.shrink_to_fit();
00070 }visaeil.clear();*/
00071 }
00072
00073 template <typename konteineris>
00074 void studentuIsskirstymas(konteineris &studentai, double &visasLaikas, konteineris &pirmunai, konteineris
&nesimokantys)
00075 {
00076     double laikasProgramos = 0.0;
00077
00078     for (int i = 0; i < 3; i++)
00079     {
00080         auto start1 = std::chrono::high_resolution_clock::now();
00081         for (Stud& j : studentai)
00082         {
00083             if (j.galutinisBalas() >= 5)
00084             {
00085                 pirmunai.push_back(j);
00086             }
00087             else
00088                 nesimokantys.push_back(j);
00089         }
00090         auto end1 = std::chrono::high_resolution_clock::now();
00091         std::chrono::duration<double> diff1 = end1 - start1;
00092         if (i != 0)
00093         {
00094             laikasProgramos += diff1.count();
00095         }
00096         if (i != 2)
00097         {
00098             pirmunai.clear();
00099             nesimokantys.clear();
00100         }
00101         if constexpr (std::is_same_v<konteineris, vector<Stud> || std::is_same_v<konteineris, deque<Stud>)
00102         {
00103             pirmunai.shrink_to_fit();
00104             nesimokantys.shrink_to_fit();
00105         }
00106     }
00107     visasLaikas += (laikasProgramos / 2);

```

```

00108     cout << "Studentu isskirstymas i dvi grupes vidutiniskai truko: " << laikasProgramos / 2 << "s" << endl;
00109 }
00110
00111 template <typename konteineris>
00112 void studentuRusiavimas(int &rusiavimasPagal, konteineris &pirmunai, konteineris &nesimokantys, double &visasLaikas)
00113 {
00114     double time = 0.0;
00115     if constexpr (std::is_same_v<konteineris, std::list<Stud> >)
00116     {
00117         for (int i = 0; i < 3; i++)
00118         {
00119             auto start = std::chrono::high_resolution_clock::now();
00120
00121             std::thread t1([&]()
00122             {
00123                 if (rusiavimasPagal == 1) pirmunai.sort(PalygintiVardas);
00124                 else if (rusiavimasPagal == 2) pirmunai.sort(PalygintiPavardes);
00125                 else if (rusiavimasPagal == 3) pirmunai.sort(PalygintiKategorijas);
00126                 else cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl; });
00127
00128             std::thread t2([&]()
00129             {
00130                 if (rusiavimasPagal == 1) nesimokantys.sort(PalygintiVardas);
00131                 else if (rusiavimasPagal == 2) nesimokantys.sort(PalygintiPavardes);
00132                 else if (rusiavimasPagal == 3) nesimokantys.sort(PalygintiKategorijas);
00133                 else cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl; });
00134
00135             t1.join();
00136             t2.join();
00137
00138             auto end = std::chrono::high_resolution_clock::now();
00139             std::chrono::duration<double> diff = end - start;
00140             if (i != 0)
00141             {
00142                 time += diff.count();
00143             }
00144         }
00145         cout << "Elementu rusiavimas vidutiniskai truko: " << time / 2 << "s" << endl;
00146         visasLaikas += (time / 2);
00147     }
00148     else
00149     {
00150         for (int j = 0; j < 3; j++)
00151         {
00152             auto start = std::chrono::high_resolution_clock::now();
00153
00154             std::thread t1([&]()
00155             {
00156                 if(rusiavimasPagal == 1) std::sort(pirmunai.begin(), pirmunai.end(), PalygintiVardas);
00157                 else if(rusiavimasPagal == 2) std::sort(pirmunai.begin(), pirmunai.end(), PalygintiPavardes);
00158                 else if(rusiavimasPagal == 3) std::sort(pirmunai.begin(), pirmunai.end(), PalygintiKategorijas);
00159                 else cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl; });
00160
00161             std::thread t2([&]()
00162             {
00163                 if(rusiavimasPagal == 1) std::sort(nesimokantys.begin(), nesimokantys.end(), PalygintiVardas);
00164                 else if(rusiavimasPagal == 2) std::sort(nesimokantys.begin(), nesimokantys.end(),
PalygintiPavardes);
00165                 else if(rusiavimasPagal == 3) std::sort(nesimokantys.begin(), nesimokantys.end(),
PalygintiKategorijas);
00166                 else cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl; });
00167
00168             t1.join();
00169             t2.join();
00170
00171             auto end = std::chrono::high_resolution_clock::now();
00172             std::chrono::duration<double> difference = end - start;
00173             if (j != 0)
00174             {
00175                 time += difference.count();
00176             }
00177         }
00178         cout << "Elementu rusiavimas vidutiniskai truko: " << time / 2 << "s" << endl;
00179         visasLaikas += (time / 2);
00180     }
00181 }
00182
00183 template <typename konteineris>
00184 void testavimasPrint(konteineris &studentai, konteineris &pirmunai, konteineris &nesimokantys, int a)
00185 {
00186     string FILEMOK = "Pirmunai" + std::to_string(a) + ".txt";
00187     string FILENESMOK = "Nesimokantys" + std::to_string(a) + ".txt";
00188     std::stringstream buferis;
00189     std::ofstream f, F;
00190
00191     f.open(FILEMOK, ios::app);
00192     f.clear();

```



```

00193     buferis < std::setw(16) < std::left < "Pavarde" < std::setw(16) < std::left < "Vardas" < std::setw(16) <
std::left < "Galutinis (Vid.)\n";
00194     buferis < "-----" < endl;
00195     for (Stud& j : studentai)//PAKEICIAU IS PIRMUNU I STUDENTUS
00196     {
00197         buferis < std::setw(16) < std::left < j.getPavarde() < std::setw(16) < std::left < j.getVardas() <
std::setw(16) < std::fixed < std::setprecision(2) < j.galutinisBalas() < endl;
00198     }
00199
00200     f < buferis.rdbuf();
00201     buferis.str("");
00202     buferis.clear();
00203     f.close();
00204     F.open(FILENESIMOK, ios::app);
00205     F.clear();
00206     buferis < std::setw(16) < std::left < "Pavarde" < std::setw(16) < std::left < "Vardas" < std::setw(16) <
std::left < "Galutinis (Vid.)\n";
00207     buferis < "-----" < endl;
00208     for (Stud& j : nesimokantys)
00209     {
00210         buferis < std::setw(16) < std::left < j.getPavarde() < std::setw(16) < std::left < j.getVardas() <
std::setw(16) < std::fixed < std::setprecision(2) < j.galutinisBalas() < endl;
00211     }
00212
00213     F < buferis.rdbuf();
00214     F.close();
00215 }
00216
00217 template <typename konteineris>
00218 void vykdomaPrograma(int &rusiavimasPagal, konteineris &studentai, konteineris &pirmunai, konteineris
&nesimokantys){
00219     for (int a = 1000; a <= 10000000; a *= 10)
00220     {
00221         studentai.clear();
00222         nesimokantys.clear();
00223         pirmunai.clear();
00224         double visasLaikas = 0.0;
00225         string testavimoFile = "Tyrimo_files\\Studentai" + std::to_string(a) + ".txt";
00226         cout < "...n"
00227             < std::to_string(a) + ".txt" < endl;
00228         nuskaitymasFile(testavimoFile, visasLaikas, studentai);
00229         studentulsskirstymas(studentai, visasLaikas, pirmunai, nesimokantys);
00230         studentuRusiavimas(rusiavimasPagal, pirmunai, nesimokantys, visasLaikas);
00231         // testavimasPrint(studentai, pirmunai, nesimokantys, a);
00232         cout < "Bendras programos vykdymo laikas: " < visasLaikas < "s" < endl;
00233     }
00234 }
00235
00236 template <typename konteineris>
00237 void studentuRusiavimas2(int rusiavimasPagal, konteineris &studentai, double &visasLaikas, double &rusiavimoLaikas)
00238 {
00239     auto start = std::chrono::high_resolution_clock::now();
00240     if constexpr (std::is_same_v<konteineris, std::list<Stud>)
00241         studentai.sort(PalygintiKategorijas);
00242     else
00243         sort(std::execution::par, studentai.begin(), studentai.end(), PalygintiKategorijas);
00244     auto end = std::chrono::high_resolution_clock::now();
00245     std::chrono::duration<double> difference = end - start;
00246
00247     cout < "Studentus rusiuoja: " < difference.count() < endl;
00248     visasLaikas += difference.count();
00249     rusiavimoLaikas += difference.count();
00250 }
00251
00252 template <typename konteineris>
00253 void studentulsskirstymas2(konteineris &studentai, double &visasLaikas, konteineris &nesimokantys, double
&rusiavimoLaikas)
00254 {
00255     auto start = std::chrono::high_resolution_clock::now();
00256     auto it = studentai.end();
00257
00258     while (true)
00259     {
00260         --it; // nes pointina ne i galutini elementa, bet i adresa po galutinio elemento
00261         if (it->galutinisBalas() < 5)
00262         {
00263             nesimokantys.push_back(*it);
00264             studentai.pop_back();
00265         }
00266         else
00267             break;
00268     }
00269     auto end = std::chrono::high_resolution_clock::now();
00270     std::chrono::duration<double> difference = end - start;
00271     if constexpr (std::is_same_v<konteineris, std::vector<Stud> ||
std::is_same_v<konteineris, std::deque<Stud>)
00272         ;
00273 }

```

```

00274     studentai.shrink_to_fit();
00275 }
00276 cout < "Studentus vidutiniskai skirste i dvi grupes: " < difference.count() < "s" < endl;
00277 visasLaikas += difference.count();
00278 auto start2 = std::chrono::high_resolution_clock::now();
00279 if constexpr (std::is_same_v<konteineris, std::list<Stud>)
00280 {
00281     nesimokantys.sort(PalygintiKategorijas);
00282 }
00283 else
00284 {
00285     sort(std::execution::par, nesimokantys.begin(), nesimokantys.end(), PalygintiKategorijas);
00286     auto end2 = std::chrono::high_resolution_clock::now();
00287     std::chrono::duration<double> difference2 = end2 - start2;
00288     cout < "Nesimokanciu konteinerio rusiavimas trukio: " < difference2.count() < "s" < endl;
00289     cout < "Bendras rusiavimo laikas: " < rusiavimoLaikas + difference2.count() < "s" < endl;
00290     visasLaikas += difference2.count();
00291 }
00292 }
00293
00294 template <typename konteineris>
00295 void vykdomaPrograma2(int rusiavimasPagal, konteineris &studentai, konteineris &nesimokantys)
00296 {
00297     for (int a = 1000; a <= 10000000; a *= 10)
00298     {
00299         studentai.clear();
00300         nesimokantys.clear();
00301         double visasLaikas = 0.0;
00302         string testavimoFile = "Tyrimo_files\\Studentai" + std::to_string(a) + ".txt";
00303         cout < "...\\n"
00304             < std::to_string(a) + ".txt" < endl;
00305         double rusiavimoLaikas = 0.0;
00306         nuskaitymasFile(testavimoFile, visasLaikas, studentai);
00307         studentuRusiavimas2(rusiavimasPagal, studentai, visasLaikas, rusiavimoLaikas);
00308         studentuIsskirstymas2(studentai, visasLaikas, nesimokantys, rusiavimoLaikas);
00309         // testavimasPrint(studentai, nesimokantys, nesimokantys, a);
00310         cout < "Bendras programos vykdymo laikas: " < visasLaikas < "s" < endl;
00311     }
00312 }
00313
00314 //template <typename konteineris>
00315 void studentuSkirstymas3(Vector<Stud> &studentai, Vector<Stud> &nesimokantys, double &visasLaikas)
00316 {
00317     nesimokantys.clear();
00318     auto start = std::chrono::high_resolution_clock::now();
00319     auto it = std::partition(studentai.begin(), studentai.end(), [](Stud &studentas){ return
00320         studentas.galutinisBalas() >= 5; });
00321     nesimokantys.resize(std::distance(it, studentai.end()));
00322     std::copy(it, studentai.end(), nesimokantys.begin());
00323     studentai.erase(it, studentai.end());
00324     auto end = std::chrono::high_resolution_clock::now();
00325     std::chrono::duration<double> diff = end - start;
00326     cout < "Studentu skirstymas i dvi grupes trukio: " < diff.count() < "s" < endl;
00327     visasLaikas += diff.count();
00328 }
00329
00330 //template <typename konteineris>
00331 void StudentuRusiavimas3(Vector<Stud> &Studentai, Vector<Stud> &nesimokantys, double &visasLaikas, int
00332 RusiuotiPagal)
00333 {
00334     /*if constexpr (std::is_same_v<konteineris, std::list<Stud>)
00335     {
00336         auto start = std::chrono::high_resolution_clock::now();
00337
00338         if (RusiuotiPagal == 1)
00339             Studentai.sort(PalygintiVardas);
00340         else if (RusiuotiPagal == 2)
00341             Studentai.sort(PalygintiPavardes);
00342         else if (RusiuotiPagal == 3)
00343             Studentai.sort(PalygintiKategorijas);
00344         else
00345             cout < "Elementu rusiavimas vidutiniskai trukio: 0.000s" < endl;
00346
00347         if (RusiuotiPagal == 1)
00348             nesimokantys.sort(PalygintiVardas);
00349         else if (RusiuotiPagal == 2)
00350             nesimokantys.sort(PalygintiPavardes);
00351         else if (RusiuotiPagal == 3)
00352             nesimokantys.sort(PalygintiKategorijas);
00353         else
00354             cout < "Elementu rusiavimas vidutiniskai trukio: 0.000s" < endl;
00355
00356         auto end = std::chrono::high_resolution_clock::now();
00357         std::chrono::duration<double> diff = end - start;
00358         cout < "Studentu rusiavimas uztrukio: " < diff.count() < endl;
00359         visasLaikas += diff.count();
00360     }
00361     */

```

```

00359     else
00360     {
00361         auto start = std::chrono::high_resolution_clock::now();
00362
00363         if (RusiuotiPagal == 1)
00364             std::sort(Studentai.begin(), Studentai.end(), PalygintiVardas);
00365         else if (RusiuotiPagal == 2)
00366             std::sort(Studentai.begin(), Studentai.end(), PalygintiPavardes);
00367         else if (RusiuotiPagal == 3)
00368             std::sort(Studentai.begin(), Studentai.end(), PalygintiKategorijas);
00369         else
00370             cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl;
00371
00372         if (RusiuotiPagal == 1)
00373             std::sort(nesimokantys.begin(), nesimokantys.end(), PalygintiVardas);
00374         else if (RusiuotiPagal == 2)
00375             std::sort(nesimokantys.begin(), nesimokantys.end(), PalygintiPavardes);
00376         else if (RusiuotiPagal == 3)
00377             std::sort(nesimokantys.begin(), nesimokantys.end(), PalygintiKategorijas);
00378         else
00379             cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl;
00380
00381         auto end = std::chrono::high_resolution_clock::now();
00382         std::chrono::duration<double> difference = end - start;
00383         cout << "Studentu rusiavimas uztruko: " << difference.count() << "s" << endl;
00384         visasLaikas += difference.count();
00385     }
00386 }
00387
00388 //template <typename konteineris>
00389 void vykdomaPrograma3(int rusiavimasPagal, Vector<Stud> &studentai, Vector<Stud> &nesimokantys)
00390 {
00391     for (int a = 1000; a <= 10000000; a *= 10)
00392     {
00393         studentai.clear();
00394         nesimokantys.clear();
00395         double visasLaikas = 0.0;
00396         string testavimoFile = "Tyrimo_files\\Studentai" + std::to_string(a) + ".txt";
00397         cout << "...\\n" << std::to_string(a) + ".txt" << endl;
00398         double rusiavimoLaikas = 0.0;
00399         nuskaitymasFile(testavimoFile, visasLaikas, studentai);
00400         studentuSkirstymas3(studentai, nesimokantys, visasLaikas);
00401         StudentuRusiavimas3(studentai, nesimokantys, visasLaikas, rusiavimasPagal);
00402         //testavimasPrint(studentai, nesimokantys, nesimokantys, a);
00403         //print(studentai, nesimokantys, rusiavimasPagal, a);
00404         cout << "Bendras programos vykdymo laikas: " << visasLaikas << "s" << endl;
00405     }
00406     //konstruktoriuTest();
00407 }
00408
00409 #endif

```

## 5.9 Strategija1.cpp File Reference

```
#include "strategija.h"
```

### Functions

- int [main](#) ()

### 5.9.1 Function Documentation

#### 5.9.1.1 main()

```
int main ()
```

## 5.10 Strategija2.cpp File Reference

```
#include "strategija.h"
```

### Functions

- int [main](#) ()

### 5.10.1 Function Documentation

#### 5.10.1.1 main()

```
int main ()
```

## 5.11 Strategija3.cpp File Reference

```
#include "strategija.h"
```

### Functions

- int [main](#) ()

### 5.11.1 Function Documentation

#### 5.11.1.1 main()

```
int main ()
```

## 5.12 unitTesting.cpp File Reference

```
#include "catch2/catch.hpp"  
#include "main.h"
```

### Macros

- #define [CATCH\\_CONFIG\\_MAIN](#)

## Functions

- [TEST\\_CASE](#) ("Copy constructor", "[Copy][constructor]")
- [TEST\\_CASE](#) ("Copy assignment", "[Copy][assignment]")
- [TEST\\_CASE](#) ("Move constructor", "[Move][constructor]")
- [TEST\\_CASE](#) ("Move assignment", "[Move][assignment]")
- [TEST\\_CASE](#) ("Destructor", "[Destructor]")
- [TEST\\_CASE](#) ("Constructor", "[Constructor]")
- [TEST\\_CASE](#) ("Getter", "[Getter]")

## 5.12.1 Macro Definition Documentation

### 5.12.1.1 CATCH\_CONFIG\_MAIN

```
#define CATCH_CONFIG_MAIN
```

## 5.12.2 Function Documentation

### 5.12.2.1 TEST\_CASE() [1/7]

```
TEST_CASE (
    "Constructor" ,
    "" [Constructor])
```

### 5.12.2.2 TEST\_CASE() [2/7]

```
TEST_CASE (
    "Copy assignment" ,
    "" [Copy][assignment])
```

### 5.12.2.3 TEST\_CASE() [3/7]

```
TEST_CASE (
    "Copy constructor" ,
    "" [Copy][constructor])
```

### 5.12.2.4 TEST\_CASE() [4/7]

```
TEST_CASE (
    "Destructor" ,
    "" [Destructor])
```

### 5.12.2.5 TEST\_CASE() [5/7]

```
TEST_CASE (
    "Getter" ,
    "" [Getter])
```

### 5.12.2.6 TEST\_CASE() [6/7]

```
TEST_CASE (
    "Move assignment" ,
    "" [Move][assignment])
```

### 5.12.2.7 TEST\_CASE() [7/7]

```
TEST_CASE (
    "Move constructor" ,
    "" [Move][constructor])
```

## 5.13 vector.h File Reference

```
#include <cstddef>
#include <limits>
#include <utility>
#include <algorithm>
#include <stdexcept>
```

### Classes

- class [Vector< T >](#)

## 5.14 vector.h

[Go to the documentation of this file.](#)

```
00001 #ifndef VECTOR_H
00002 #define VECTOR_H
00003
00004 #include <cstddef>
00005 #include <limits>
00006 #include <utility>
00007 #include <algorithm>
00008 #include <stdexcept>
00009
00010 template <typename T>
00011 class Vector
00012 {
00013 private:
00014     T *vec;
00015     size_t capacity;
00016     size_t size;
00017
00018 public:
00019     Vector() : vec(nullptr), capacity(0), size(0) {}
00020     Vector(size_t Size) : size(Size), capacity(Size)
00021     {
00022         vec = new T[capacity];
00023     }
00024
00025     Vector(std::initializer_list<T> list) : size(list.size()), capacity(list.size())
00026     {
00027         vec = new T[capacity];
00028         std::copy(list.begin(), list.end(), vec);
00029     }
00030
00031     ~Vector()
00032     {
```

```

00033         capacity = 0;
00034         size = 0;
00035         delete[] vec;
00036         vec=nullptr;
00037     }
00038
00039     // copy constructor
00040     Vector(const Vector &v) : size(v.size), capacity(v.capacity)
00041     {
00042         vec = new T[capacity];
00043         std::copy(v.vec, v.vec + size, vec);
00044     }
00045
00046     // copy assignment operator
00047     Vector &operator=(const Vector &v)
00048     {
00049         if (this != &v)
00050         {
00051             delete[] vec;
00052             size = v.size;
00053             capacity = v.capacity;
00054             vec = new T[capacity];
00055             std::copy(v.vec, v.vec + size, vec);
00056         }
00057         return *this;
00058     }
00059
00060     // move constructor
00061     Vector(Vector &&v) : vec(std::move(v.vec)), capacity(std::move(v.capacity)), size(std::move(v.size))
00062     {
00063         v.vec = nullptr;
00064         v.size = 0;
00065         v.capacity = 0;
00066     }
00067
00068     bool operator==(const Vector &vv) const
00069     {
00070         if (size != vv.Size())
00071             return false;
00072         for (size_t i = 0; i < size; i++)
00073         {
00074             if (vec[i] != vv[i])
00075             {
00076                 return false;
00077             }
00078         }
00079         return true;
00080     }
00081
00082     bool operator!=(const Vector &vv) const
00083     {
00084         return !(*this == vv);
00085     }
00086
00087     Vector &operator=(Vector &&v) noexcept
00088     {
00089         if (this != &v)
00090         {
00091             delete[] vec;
00092             vec = v.vec;
00093             size = v.size;
00094             capacity = v.capacity;
00095             v.vec = nullptr;
00096             v.size = 0;
00097             v.capacity = 0;
00098         }
00099         return *this;
00100     }
00101
00102     size_t Size() const { return this->size; }
00103
00104     T *Data() const { return vec; }
00105
00106     size_t Capacity() const { return this->capacity; }
00107
00108     size_t max_size() { return std::numeric_limits<T>::max() / sizeof(T); }
00109
00110     bool empty() { return size == 0; }
00111
00112     void assign(size_t amount, T &value)
00113     {
00114         if (amount > capacity)
00115         {
00116             resize(amount);
00117         }
00118         std::fill_n(vec, amount, value);
00119         size = amount;

```

```

00120     }
00121
00122     void assign(T *begin, T *end)
00123     {
00124         size_t amount = end - begin;
00125         if (amount > capacity)
00126         {
00127             resize(amount);
00128         };
00129         std::copy(begin, end, vec);
00130         size = amount;
00131     }
00132
00133     void assign(std::initializer_list<T> list)
00134     {
00135         size = list.size();
00136         if (size > capacity)
00137         {
00138             resize(size);
00139         }
00140         std::copy(list.begin(), list.end(), vec);
00141     }
00142
00143     void append_range(std::initializer_list<T> list)
00144     { // PAKEIST
00145         if (size + list.size() > capacity)
00146         {
00147             reserve(size + list.size());
00148         }
00149         std::copy(list.begin(), list.end(), vec + size);
00150         size += list.size();
00151     }
00152
00153     T &front() { return vec[0]; }
00154
00155     T &back() { return vec[size - 1]; }
00156
00157     T &operator[](size_t index)
00158     {
00159         if (index > size)
00160         {
00161             throw std::out_of_range("index out of range");
00162         }
00163         return vec[index];
00164     }
00165     const T &operator[](size_t index) const
00166     {
00167         if (index >= size)
00168             throw std::out_of_range("index out of range");
00169         return vec[index];
00170     }
00171
00172     T *end() { return vec + size; }
00173     T *begin() { return vec; }
00174
00175     const T *cbegin() const { return vec; }
00176     const T *cend() const { return vec + size; }
00177
00178     T *rbegin() { return size == 0 ? nullptr : vec + size - 1; }
00179     T *rend() { return size == 0 ? nullptr : vec - 1; }
00180
00181     const T *crbegin() const { return size == 0 ? nullptr : vec + size - 1; }
00182     const T *crend() const { return size == 0 ? nullptr : vec - 1; }
00183
00184     void push_back(const T &value)
00185     {
00186         if (size >= capacity)
00187         {
00188             reserve(capacity == 0 ? 1 : capacity * 2);
00189         }
00190         vec[size++] = value;
00191     }
00192
00193     void pop_back()
00194     {
00195         if (size == 0)
00196         {
00197             return;
00198         }
00199         --size;
00200     }
00201
00202     T &at(size_t index)
00203     {
00204         if (index >= size)
00205             throw std::out_of_range("index out of range");
00206         return vec[index];

```



```

00207     }
00208
00209     void clear()
00210     {
00211         delete[] vec;
00212         vec = nullptr;
00213         size = 0;
00214         capacity = 0;
00215     }
00216
00217     void shrink_to_fit()
00218     { // siaip su resize galejau
00219         if (size == capacity)
00220         {
00221             return;
00222         }
00223         T *Nvec = new T[size];
00224         std::move(vec, vec + size, Nvec);
00225         delete[] vec;
00226         vec = Nvec;
00227         capacity = size;
00228     }
00229
00230     void swap(Vector &v)
00231     {
00232         std::swap(size, v.size);
00233         std::swap(capacity, v.capacity);
00234         std::swap(vec, v.vec);
00235     }
00236
00237     void reserve(size_t Ncap)
00238     {
00239         if (Ncap > capacity)
00240         {
00241             T *Nvec = new T[Ncap];
00242             std::copy(vec, vec + size, Nvec);
00243             delete[] vec;
00244             vec = Nvec;
00245             capacity = Ncap;
00246         }
00247     }
00248
00249     void insert(T *index, const T &element)
00250     {
00251         size_t position = index - vec;
00252         if (size == capacity)
00253         {
00254             reserve(capacity == 0 ? 1 : capacity * 2);
00255         }
00256         for (auto i = size; i > position; i--)
00257         {
00258             vec[i] = vec[i - 1];
00259         }
00260         vec[position] = element;
00261         size++;
00262     }
00263
00264     void erase(T *index)
00265     {
00266         size_t position = index - vec;
00267         if (index < vec || index >= vec + size)
00268             throw std::out_of_range("out of range");
00269         for (size_t i = position; i + 1 < size; i++)
00270         {
00271             vec[i] = vec[i + 1];
00272         }
00273         size--;
00274     }
00275
00276     void erase(T *begin, T *end)
00277     {
00278         size_t amount = end - begin;
00279         size_t index = begin - vec;
00280         for (size_t i = index; i + amount < size; i++)
00281         {
00282             vec[i] = std::move(vec[i + amount]);
00283         }
00284         size -= amount;
00285     }
00286
00287     void resize(size_t newCap)
00288     {
00289         if (newCap < size)
00290         {
00291             size = newCap;

```

```

00294     }
00295     else
00296         capacity = newCap;
00297     T *Nvec = new T[capacity];
00298     std::copy(vec, vec + size, Nvec);
00299     delete[] vec;
00300     vec = Nvec;
00301 }
00302
00303 void resize(size_t newCap, const T &element)
00304 {
00305     if (newCap < size)
00306     {
00307         size = newCap;
00308     }
00309     capacity = newCap;
00310     T *Nvec = new T[capacity];
00311     std::copy(vec, vec + size, Nvec);
00312     for (size_t i = size; i < capacity; ++i)
00313     {
00314         Nvec[i] = element;
00315     }
00316     delete[] vec;
00317     vec = Nvec;
00318     size = newCap;
00319 }
00320
00321 bool operator<(const Vector &v) const
00322 {
00323     size_t minSize = std::min(size, v.size);
00324     for (size_t i = 0; i < minSize; ++i)
00325     {
00326         if (vec[i] < v.vec[i])
00327             return true;
00328         if (vec[i] > v.vec[i])
00329             return false;
00330     }
00331     return size < v.size;
00332 }
00333
00334 bool operator<=(const Vector &v) const
00335 {
00336     return !(*this > v);
00337 }
00338
00339 bool operator>(const Vector &v) const
00340 {
00341     return v < *this;
00342 }
00343
00344 bool operator>=(const Vector &v) const
00345 {
00346     return !(*this < v);
00347 }
00348 };
00349
00350 #endif

```

## 5.15 vectorUnitTests.cpp File Reference

```

#include "../catch2/catch.hpp"
#include "vector.h"
#include "iostream"

```

### Macros

- `#define CATCH_CONFIG_MAIN`

### Functions

- `TEST_CASE ("Constructor", "[Constructor]")`

- [TEST\\_CASE](#) ("Copy constructor", "[Copy][constructor]")
- [TEST\\_CASE](#) ("Copy assignment", "[Copy][assignment]")
- [TEST\\_CASE](#) ("Move constructor", "[Move][constructor]")
- [TEST\\_CASE](#) ("Move assignment", "[Move][assignment]")
- [TEST\\_CASE](#) ("push back", "[push][back]")
- [TEST\\_CASE](#) ("pop back", "[pop][back]")
- [TEST\\_CASE](#) ("front back", "[front][back]")
- [TEST\\_CASE](#) ("append range", "[append][range]")
- [TEST\\_CASE](#) ("swap", "[swap]")
- [TEST\\_CASE](#) ("erase", "[erase]")
- [TEST\\_CASE](#) ("empty", "[empty]")
- [TEST\\_CASE](#) ("shrink to fit", "[shrink][fit]")
- [TEST\\_CASE](#) ("assign", "[assign]")
- [TEST\\_CASE](#) ("assign range", "[assign][range]")
- [TEST\\_CASE](#) ("assign from list", "[assign][list]")
- [TEST\\_CASE](#) ("insert element", "[insert][element]")
- [TEST\\_CASE](#) ("operators", "[operators]")
- [TEST\\_CASE](#) ("rbegin rend", "[rbegin][rend]")
- [TEST\\_CASE](#) ("resize", "[resize]")

## 5.15.1 Macro Definition Documentation

### 5.15.1.1 CATCH\_CONFIG\_MAIN

```
#define CATCH_CONFIG_MAIN
```

## 5.15.2 Function Documentation

### 5.15.2.1 TEST\_CASE() [1/20]

```
TEST_CASE (
    "append range" ,
    "" [append][range])
```

### 5.15.2.2 TEST\_CASE() [2/20]

```
TEST_CASE (
    "assign from list" ,
    "" [assign][list])
```

### 5.15.2.3 TEST\_CASE() [3/20]

```
TEST_CASE (
    "assign range" ,
    "" [assign][range])
```

**5.15.2.4 TEST\_CASE()** [4/20]

```
TEST_CASE (
    "assign" ,
    "" [assign])
```

**5.15.2.5 TEST\_CASE()** [5/20]

```
TEST_CASE (
    "Constructor" ,
    "" [Constructor])
```

**5.15.2.6 TEST\_CASE()** [6/20]

```
TEST_CASE (
    "Copy assignment" ,
    "" [Copy][assignment])
```

**5.15.2.7 TEST\_CASE()** [7/20]

```
TEST_CASE (
    "Copy constructor" ,
    "" [Copy][constructor])
```

**5.15.2.8 TEST\_CASE()** [8/20]

```
TEST_CASE (
    "empty" ,
    "" [empty])
```

**5.15.2.9 TEST\_CASE()** [9/20]

```
TEST_CASE (
    "erase" ,
    "" [erase])
```

**5.15.2.10 TEST\_CASE()** [10/20]

```
TEST_CASE (
    "front back" ,
    "" [front][back])
```

**5.15.2.11 TEST\_CASE()** [11/20]

```
TEST_CASE (
    "insert element" ,
    "" [insert][element])
```

**5.15.2.12 TEST\_CASE()** [12/20]

```
TEST_CASE (
    "Move assignment" ,
    "" [Move][assignment])
```

**5.15.2.13 TEST\_CASE()** [13/20]

```
TEST_CASE (
    "Move constructor" ,
    "" [Move][constructor])
```

**5.15.2.14 TEST\_CASE()** [14/20]

```
TEST_CASE (
    "operators" ,
    "" [operators])
```

**5.15.2.15 TEST\_CASE()** [15/20]

```
TEST_CASE (
    "pop back" ,
    "" [pop][back])
```

**5.15.2.16 TEST\_CASE()** [16/20]

```
TEST_CASE (
    "push back" ,
    "" [push][back])
```

**5.15.2.17 TEST\_CASE()** [17/20]

```
TEST_CASE (
    "rbegin rend" ,
    "" [rbegin][rend])
```

**5.15.2.18 TEST\_CASE()** [18/20]

```
TEST_CASE (
    "resize" ,
    "" [resize])
```

**5.15.2.19 TEST\_CASE()** [19/20]

```
TEST_CASE (
    "shrink to fit" ,
    "" [shrink][fit])
```

**5.15.2.20 TEST\_CASE()** [20/20]

```
TEST_CASE (
    "swap" ,
    "" [swap])
```



# Index

- ~Stud
  - Stud, [10](#)
- ~Vector
  - Vector< T >, [15](#)
- ~zmogus
  - zmogus, [21](#)
- append\_range
  - Vector< T >, [15](#)
- assign
  - Vector< T >, [15](#), [16](#)
- at
  - Vector< T >, [16](#)
- back
  - Vector< T >, [16](#)
- begin
  - Vector< T >, [16](#)
- Capacity
  - Vector< T >, [16](#)
- CATCH\_CONFIG\_MAIN
  - unitTesting.cpp, [47](#)
  - vectorUnitTests.cpp, [53](#)
- cbegin
  - Vector< T >, [16](#)
- cend
  - Vector< T >, [16](#)
- clear
  - Stud, [11](#)
  - Vector< T >, [16](#)
- constrTest
  - functions.cpp, [24](#)
- copyAssignTest
  - functions.cpp, [24](#)
- copyConstrTest
  - functions.cpp, [24](#)
- crbegin
  - Vector< T >, [17](#)
- crend
  - Vector< T >, [17](#)
- Data
  - Vector< T >, [17](#)
- destructorTest
  - functions.cpp, [24](#)
- duomenu\_generavimas
  - functions.cpp, [24](#)
  - functions.h, [28](#)
- duomenu\_ivedimas
  - functions.cpp, [24](#)
  - functions.h, [28](#)
- empty
  - Vector< T >, [17](#)
- end
  - Vector< T >, [17](#)
- erase
  - Vector< T >, [17](#)
- filePasirinkimas
  - functions.cpp, [24](#)
  - functions.h, [28](#)
- fileskait
  - functions.h, [28](#)
- front
  - Vector< T >, [17](#)
- functions.cpp, [23](#)
  - constrTest, [24](#)
  - copyAssignTest, [24](#)
  - copyConstrTest, [24](#)
  - destructorTest, [24](#)
  - duomenu\_generavimas, [24](#)
  - duomenu\_ivedimas, [24](#)
  - filePasirinkimas, [24](#)
  - GeneruotiFiles, [24](#)
  - inputTest, [24](#)
  - konstruktoriuTest, [24](#)
  - moveAssignTest, [25](#)
  - moveConstrTest, [25](#)
  - operator<<, [25](#)
  - operator>>, [25](#)
  - outputTest, [25](#)
  - PalygintiKategorijas, [25](#)
  - PalygintiPavardes, [25](#)
  - PalygintiVardas, [25](#)
  - paz\_gener, [26](#)
  - pazymiu\_ivedimas, [26](#)
  - print, [26](#)
  - printCases, [26](#)
  - pushbackTest, [26](#)
  - tikrinimas, [26](#)
  - tinkamas\_char, [26](#)
  - tinkamas\_int, [26](#)
  - vardai, [27](#)
- functions.h, [27](#)
  - duomenu\_generavimas, [28](#)
  - duomenu\_ivedimas, [28](#)
  - filePasirinkimas, [28](#)
  - fileskait, [28](#)

- GeneruotiFiles, 28
- konstruktoriuTest, 28
- PalygintiKategorijas, 28
- PalygintiPavardes, 28
- PalygintiVardas, 28
- paz\_gener, 29
- pazymiu\_ivedimas, 29
- printCases, 29
- PrintVektorius, 29
- pushbackTest, 29
- tikrinimas, 29
- tinkamas\_char, 29
- tinkamas\_int, 30
- tyrimai, 30
- vardai, 30
- vectorIdejimas, 30
- galutinis
  - Stud, 11
- galutinisBalas
  - Stud, 11
- GeneruotiFiles
  - functions.cpp, 24
  - functions.h, 28
- getEgzaminas
  - Stud, 11
- getND
  - Stud, 11
- getPavarde
  - zmogus, 22
- getPaz
  - Stud, 11
- getVardas
  - zmogus, 22
- grazintiPaskutini
  - Stud, 11
- inputTest
  - functions.cpp, 24
- insert
  - Vector< T >, 18
- konstruktoriuTest
  - functions.cpp, 24
  - functions.h, 28
- konteinerisDeque
  - main.h, 35
- konteinerisList
  - main.h, 35
- konteinerisVector
  - main.h, 35
- main
  - main.cpp, 34
  - Strategija1.cpp, 45
  - Strategija2.cpp, 46
  - Strategija3.cpp, 46
- main.cpp, 34
  - main, 34
- main.h, 35
  - konteinerisDeque, 35
  - konteinerisList, 35
  - konteinerisVector, 35
- max\_size
  - Vector< T >, 18
- moveAssignTest
  - functions.cpp, 25
- moveConstTest
  - functions.cpp, 25
- nuskaitymasFile
  - strategija.h, 38
- operator!=
  - Stud, 11
  - Vector< T >, 18
- operator<
  - Vector< T >, 18
- operator<<
  - functions.cpp, 25
  - Stud, 13
- operator<=
  - Vector< T >, 18
- operator>
  - Vector< T >, 19
- operator>>
  - functions.cpp, 25
  - Stud, 13
- operator>=
  - Vector< T >, 19
- operator=
  - Stud, 11, 12
  - Vector< T >, 18
  - zmogus, 22
- operator==
  - Stud, 12
  - Vector< T >, 18
- operator[]
  - Vector< T >, 19
- outputTest
  - functions.cpp, 25
- PalygintiKategorijas
  - functions.cpp, 25
  - functions.h, 28
- PalygintiPavardes
  - functions.cpp, 25
  - functions.h, 28
- PalygintiVardas
  - functions.cpp, 25
  - functions.h, 28
- pavarde
  - zmogus, 22
- paz\_gener
  - functions.cpp, 26
  - functions.h, 29
- pazKiekis
  - Stud, 12



pazymiu\_ivedimas  
    functions.cpp, 26  
    functions.h, 29  
pop\_back  
    Vector< T >, 19  
print  
    functions.cpp, 26  
printCases  
    functions.cpp, 26  
    functions.h, 29  
PrintVektorius  
    functions.h, 29  
push\_back  
    Vector< T >, 19  
pushbackTest  
    functions.cpp, 26  
    functions.h, 29  
  
rbegin  
    Vector< T >, 19  
removeLast  
    Stud, 12  
removeND  
    Stud, 12  
rend  
    Vector< T >, 19  
reserve  
    Vector< T >, 20  
resize  
    Vector< T >, 20  
  
setBalasGalutinisVid  
    Stud, 12  
setEgzaminas  
    Stud, 12  
setND  
    Stud, 12  
setPavarde  
    Stud, 12  
    zmogus, 22  
setVardas  
    Stud, 13  
    zmogus, 22  
shrink\_to\_fit  
    Vector< T >, 20  
Size  
    Vector< T >, 20  
strategija.h, 38  
    nuskaitymasFile, 38  
    studentuIsskirstymas, 38  
    studentuIsskirstymas2, 39  
    studentuRusiavimas, 39  
    studentuRusiavimas2, 39  
    StudentuRusiavimas3, 39  
    studentuSkirstymas3, 39  
    testavimasPrint, 39  
    vykdomaPrograma, 40  
    vykdomaPrograma2, 40  
    vykdomaPrograma3, 40  
  
Strategija1.cpp, 45  
    main, 45  
Strategija2.cpp, 46  
    main, 46  
Strategija3.cpp, 46  
    main, 46  
Stud, 9  
    ~Stud, 10  
    clear, 11  
    galutinis, 11  
    galutinisBalas, 11  
    getEgzaminas, 11  
    getND, 11  
    getPaz, 11  
    grazintiPaskutini, 11  
    operator!=, 11  
    operator<<, 13  
    operator>>, 13  
    operator=, 11, 12  
    operator==, 12  
    pazKiekis, 12  
    removeLast, 12  
    removeND, 12  
    setBalasGalutinisVid, 12  
    setEgzaminas, 12  
    setND, 12  
    setPavarde, 12  
    setVardas, 13  
    Stud, 10  
studentuIsskirstymas  
    strategija.h, 38  
studentuIsskirstymas2  
    strategija.h, 39  
studentuRusiavimas  
    strategija.h, 39  
studentuRusiavimas2  
    strategija.h, 39  
StudentuRusiavimas3  
    strategija.h, 39  
studentuSkirstymas3  
    strategija.h, 39  
swap  
    Vector< T >, 20  
  
TEST\_CASE  
    unitTesting.cpp, 47, 48  
    vectorUnitTests.cpp, 53–55  
testavimasPrint  
    strategija.h, 39  
tikrinimas  
    functions.cpp, 26  
    functions.h, 29  
tinkamas\_char  
    functions.cpp, 26  
    functions.h, 29  
tinkamas\_int  
    functions.cpp, 26  
    functions.h, 30  
tyrimai

- functions.h, 30
- unitTesting.cpp, 46
  - CATCH\_CONFIG\_MAIN, 47
  - TEST\_CASE, 47, 48
- vardai
  - functions.cpp, 27
  - functions.h, 30
- vardas
  - zmogus, 22
- Vector
  - Vector< T >, 14, 15
- Vector< T >, 13
  - ~Vector, 15
  - append\_range, 15
  - assign, 15, 16
  - at, 16
  - back, 16
  - begin, 16
  - Capacity, 16
  - cbegin, 16
  - cend, 16
  - clear, 16
  - crbegin, 17
  - crend, 17
  - Data, 17
  - empty, 17
  - end, 17
  - erase, 17
  - front, 17
  - insert, 18
  - max\_size, 18
  - operator!=, 18
  - operator<, 18
  - operator<=, 18
  - operator>, 19
  - operator>=, 19
  - operator=, 18
  - operator==, 18
  - operator[], 19
  - pop\_back, 19
  - push\_back, 19
  - rbegin, 19
  - rend, 19
  - reserve, 20
  - resize, 20
  - shrink\_to\_fit, 20
  - Size, 20
  - swap, 20
  - Vector, 14, 15
- vector.h, 48
- vectorIdejimas
  - functions.h, 30
- vectorUnitTests.cpp, 52
  - CATCH\_CONFIG\_MAIN, 53
  - TEST\_CASE, 53–55
- vykdomaPrograma
  - strategija.h, 40
- vykdomaPrograma2
  - strategija.h, 40
- vykdomaPrograma3
  - strategija.h, 40
- zmogus, 21
  - ~zmogus, 21
  - getPavarde, 22
  - getVardas, 22
  - operator=, 22
  - pavarde, 22
  - setPavarde, 22
  - setVardas, 22
  - vardas, 22
  - zmogus, 21